

Subject Code	Subject Name	Category	L	T	P	S	Credits	Inst.Hours	Marks		
									CIA	External	Total
23BCE1FC	Problem Solving Techniques	FC	2	-	-	-	2	2	25	75	100
Learning Objectives											
LO1	Familiarize with writing of algorithms, fundamentals of C and philosophy of problem Solving.										
LO2	Implement different programming constructs and decomposition of problems into Functions.										
LO3	Use data flow diagram, Pseudo code to implement solutions.										
LO4	Define and use of arrays with simple applications										
LO5	Understand about operating system and their uses										
	Contents										
UNIT I	Introduction: History, characteristics and limitations of Computer. Hardware/Anatomy of Computer: CPU, Memory, Secondary storage devices, Input Devices and Output devices. Types of Computers: PC, Workstation, Minicomputer, Main frame and Supercomputer. Software: System software and Application software. Programming Languages: Machine language, Assembly language, High-level language, 4GL and 5GL-Features of good programming language. Translators: Interpreters and Compilers.										
UNIT II	Data: Data types, Input, Processing of data, Arithmetic Operators, Hierarchy of operations and Output. Different phases in Program Development Cycle (PDC). Structured Programming: Algorithm: Features of good algorithm, Benefits and drawbacks of algorithm. Flowcharts: Advantages and limitations of flowcharts, when to use flowcharts, flowchart symbols and types of flow charts. Pseudo code: Writing a pseudo code. Coding, documenting and testing a program: Comment lines and types of errors. Program design: Modular Programming.										
UNIT III	Selection Structures: Relational and Logical Operators -Selecting from Several Alternatives – Applications of Selection Structures. Repetition Structures: Counter Controlled Loops–Nested Loops–Applications of Repetition Structures.										
UNIT IV	Data: Numeric Data and Character Based Data. Arrays: One Dimensional Array - Two Dimensional Arrays – Strings as Arrays of Characters.										
UNIT V	Data Flow Diagrams: Definition, DFD symbols and types of DFDs. Program Modules: Subprograms-Value and Reference parameters-Scope of a variable-Functions–Recursion. Files: File Basics-Creating and reading a sequential file-Modifying Sequential Files.										
Course Outcomes									Programme Outcomes		

UNIT I

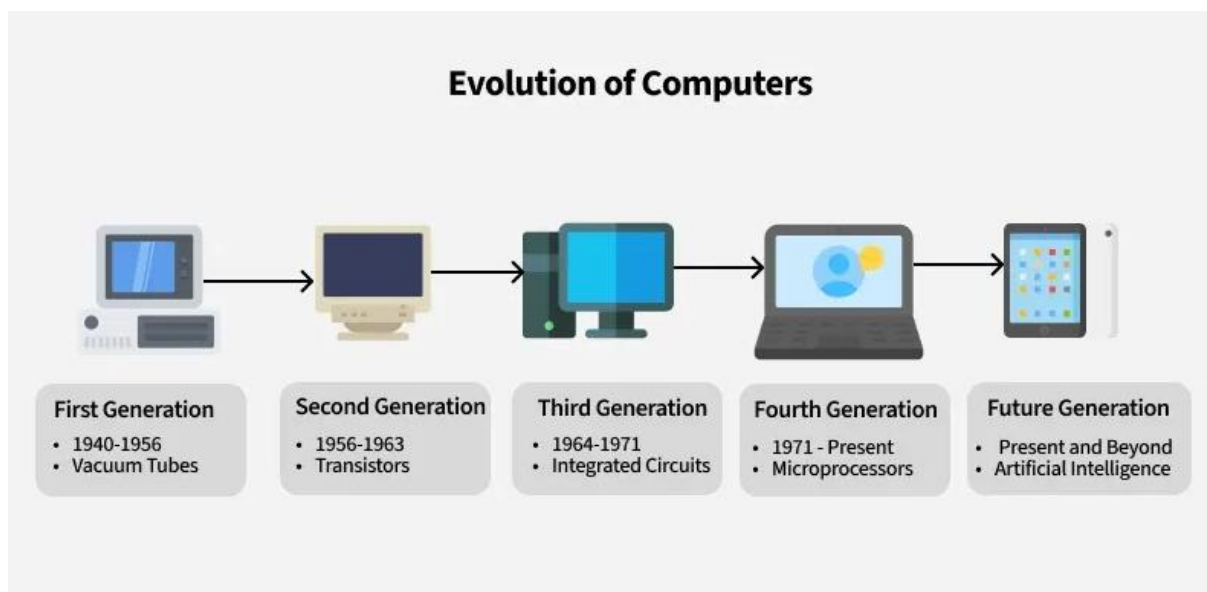
Introduction:

A **computer** is an electronic device that accepts data as input, processes it according to a set of instructions, and produces meaningful output (information).

History

The Evolution of Computers

The history of computers spans thousands of years, from early counting devices to the powerful systems we use today. Here's an overview of the key milestones in the evolution of computers:



First Generation Computers (1940–1956)

Main Features:

- Used **vacuum tubes** as the main components for CPU and memory.
- Very **large in size**, **slow in speed**, and **high in cost**.
- Consumed **a lot of electricity** and generated much heat.
- **Batch processing systems** and **punch cards** were used for input and output.
- **Magnetic tape and paper tape** served as storage media.
- Programming was done using **machine language**.

Examples:

ENIAC, UNIVAC, EDSAC, EDVAC.

Second Generation Computers (1957–1963)

Main Features:

- Used **transistors** instead of vacuum tubes.

- **Smaller, faster, cheaper, and more reliable** than first-generation computers.
- Consumed **less power** and generated **less heat**.
- **Magnetic cores** were used for primary memory.
- **Magnetic disks and tapes** were used for secondary storage.
- **Batch processing** and **multiprogramming** operating systems were introduced.
- Programming was done using **Assembly language, COBOL, and FORTRAN**.

Examples:

IBM 1620, IBM 7094, CDC 1604, CDC 3600.

Third Generation Computers (1964–1971)

Main Features:

- Used **Integrated Circuits (ICs)** instead of transistors.
- One IC contained **many transistors**, which made computers more powerful and compact.
- Increased **processing speed** and **efficiency**.
- **Reduced size, cost, and maintenance**.
- Supported **time-sharing, remote processing, and multiprogramming**.
- Used **high-level programming languages** such as FORTRAN-II to IV, COBOL, and PASCAL.

Examples:

IBM 360 Series, Honeywell 6000 Series, IBM 370/168.

Fourth Generation Computers (1971–1980)

Main Features:

- Used **Very Large Scale Integration (VLSI)** circuits.
- Each chip contained **millions of transistors** and circuit elements.
- Became **smaller, faster, powerful, reliable, and cheaper**.
- Used **microprocessors**, which combined all CPU functions on a single chip.
- Supported **real-time, time-sharing, and distributed** operating systems.
- Used **C and C++** programming languages.

Examples:

STAR 1000, PDP 11, CRAY-1, CRAY-X-MP.

Fifth Generation Computers (1980–Present)

Main Features:

- Use **Ultra Large Scale Integration (ULSI)** technology.
- Contain **millions of electronic components** on a single microprocessor chip.

- Introduced **Artificial Intelligence (AI)** and **machine learning** capabilities.
- Support **parallel processing**, **cloud computing**, and **internet connectivity**.
- Have **high speed**, **huge storage**, and **user-friendly interfaces**.
- Use modern languages like **C**, **C++**, **Java**, **Python**, and **.NET**.

Examples:

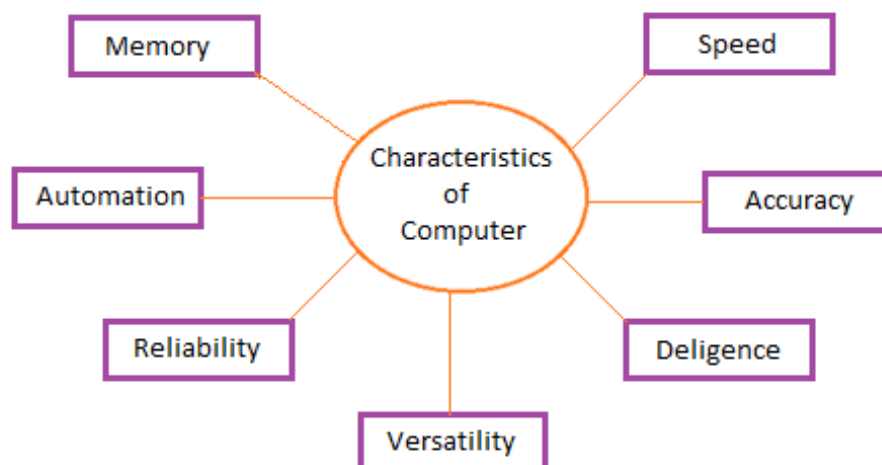
Desktop Computers, Laptops, Notebooks, Ultrabooks, Supercomputers.

Summary Table

Generation	Period	Main Technology	Languages Used	Examples
First	1940–1956	Vacuum Tubes	Machine Language	ENIAC, UNIVAC
Second	1957–1963	Transistors	COBOL, FORTRAN	IBM 7094, CDC 1604
Third	1964–1971	Integrated Circuits	FORTRAN, PASCAL	IBM 360
Fourth	1971–1980	VLSI	C, C++	CRAY-1, PDP 11
Fifth	1980–Present	ULSI, AI	Java, Python, .NET	Laptops, Desktops

Characteristics

- A **computer** is an **electronic device** that accepts data as **input**, processes it according to given **instructions**, and produces **output** in a meaningful form.
- It can **store**, **retrieve**, and **process data** quickly and accurately. A computer helps in performing both simple and complex tasks efficiently.



Characteristics of Computer System

1. Speed

- ❖ Executing mathematical calculation, a computer works faster and more accurately than human.
- ❖ Computers have the ability to process so many millions (1,000,000) of instructions per second. Computer operations are performed in micro and nano seconds.
- ❖ A computer is a time saving device. It performs several calculations and tasks in few seconds that we take hours to solve.
- ❖ The speed of a computer is measure in terms of GigaHertz and MegaHertz.

2. Diligence

- ❖ A human cannot work for several hours without resting, yet a computer never tires.
- ❖ A computer can conduct millions of calculations per second with complete precision without stopping.
- ❖ A computer can consistently and accurately do millions of jobs or calculations. There is no weariness or lack of concentration. Its memory ability also places it ahead of humans.

3. Reliability

- ❖ A computer is reliable. The output results never differ unless the input varies. the output is totally depend on the input.
- ❖ When an input is the same the output will also be the same.
- ❖ A computer produces consistent results for similar sets of data, if we provide the same set of input at any time we will get the same result.

4. Automation

- ❖ The world is quickly moving toward AI (Artificial Intelligence)-based technology. A computer may conduct tasks automatically after instructions are programmed.
- ❖ By executing jobs automatically, this computer feature replaces thousands of workers.
- ❖ Automation in computing is often achieved by the use of a program, a script, or batch processing.

5. Versatility

- ❖ Versatility refers to a capacity of computer. Computer performs different types of tasks with the same accuracy and efficiency.
- ❖ A computer can perform multiple tasks at the same time this is known as versatility.

- ❖ For example, while listening to music, we may develop our project using PowerPoint and Wordpad, or we can design a website.

6. Memory

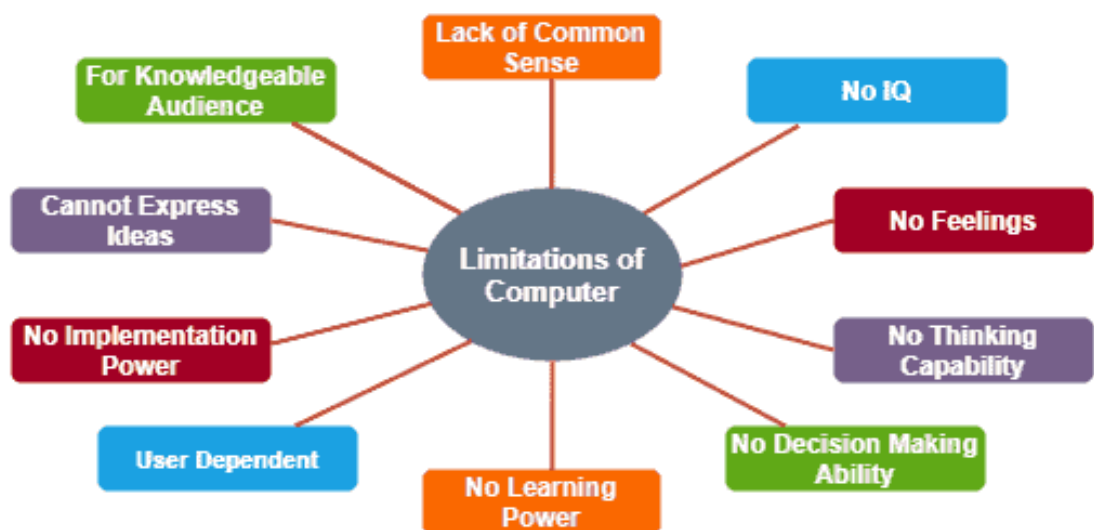
- ❖ A computer can store millions of records. these records may be accessed with complete precision.
- ❖ Computer memory storage_capacity is measured in Bytes, Kilobytes(KB), Megabytes(MB), Gigabytes(GB), and Terabytes(TB).
- ❖ A computer has built-in memory known as primary memory.

7. Accuracy

- ❖ When a computer performs a computation or operation, the chances of errors occurring are low.
- ❖ Errors in a computer are caused by human's submitting incorrect data.
- ❖ A computer can do a variety of operations and calculations fast and accurately.

Limitations of Computer

A computer is a powerful and fast machine, but it is **not intelligent**, **cannot feel emotions**, and **depends on human input**. It only works based on the **instructions given**, and if the input is wrong, the result will also be wrong.



Limitations of Computer

1. No Intelligence

- A computer **does not have its own intelligence**.

- It can only **follow the instructions** given by the user or programmer.
- Computers cannot **think, reason, or learn** from experience like humans do.
- They work only according to the **programs and data** fed into them.

2. No Emotions

- Computers are **emotionless machines**.
- They do not have **feelings, moods, or attitudes**.
- Unlike humans, they cannot express **sympathy, anger, or happiness**.
- This makes them efficient but also limits their ability to **understand human emotions**.

3. Dependence on Humans

- Computers **depend entirely on humans** to operate.
- They require **instructions, data, and programs** to perform any task.
- Without human guidance, a computer **cannot start, stop, or make decisions**.
- All computer operations are based on **human input and control**.

4. Limited Decision-Making Ability

- Computers can make decisions only based on **predefined conditions** or logic.
- They cannot make **judgments or creative decisions** like humans.
- If a situation arises outside its programming, the computer **cannot act intelligently**.

5. Garbage in – Garbage out (GIGO)

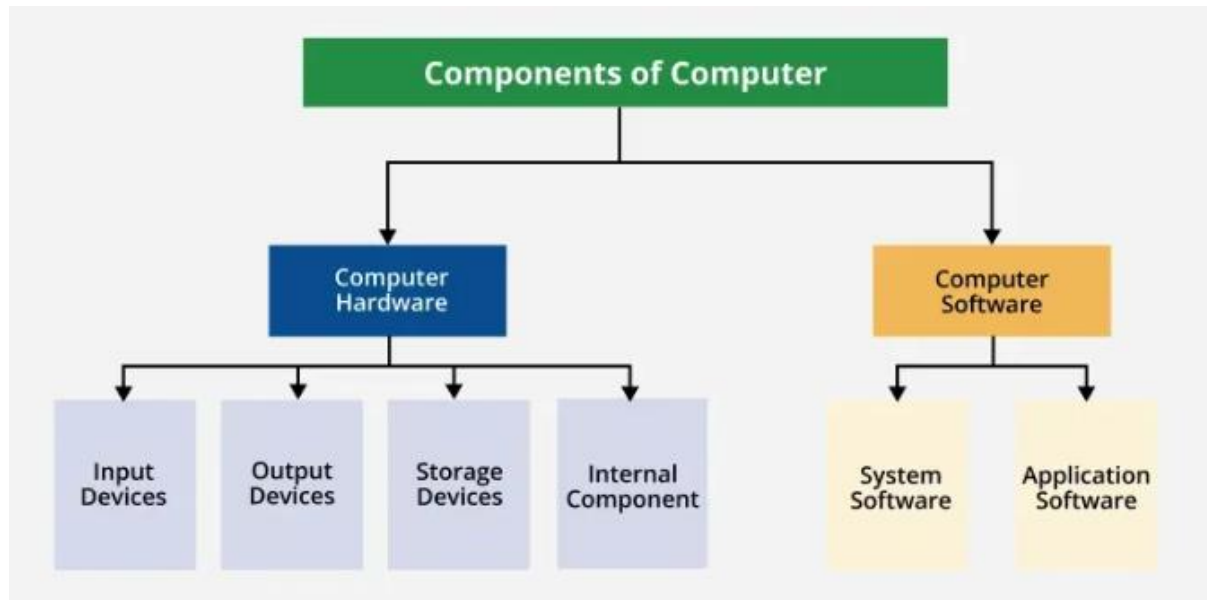
- The quality of the computer's output **depends on the quality of the input**.
- If **incorrect data** or **faulty instructions** are given, the output will also be **wrong**.
- This principle is known as **GIGO — Garbage In, Garbage Out**.
- Example: If wrong marks are entered in a grading system, the computer will still calculate based on those wrong inputs.

Summary Table

Limitation	Description
No Intelligence	Works only as programmed, cannot think on its own
No Emotions	Cannot feel or express emotions
Dependence on Humans	Needs instructions to operate
Limited Decision-Making	Cannot make creative or logical judgments
GIGO	Wrong input gives wrong output

Hardware/Anatomy of Computer

Computer hardware refers to the **physical components** of a computer that you can see and touch. These components work together to process input and deliver output based on user instructions. In this explore the different types of computer hardware, their functions, and they interact to make your computer work.



The computer has mainly had two major components:

1. **Hardware**
2. **Software**



What is a Computer Hardware

Computer hardware is a physical device of computers that we can see and touch. E.g. Monitor, Central Processing Unit, Mouse, Joystick, etc. Using these devices, we can control computer operations like input and output.

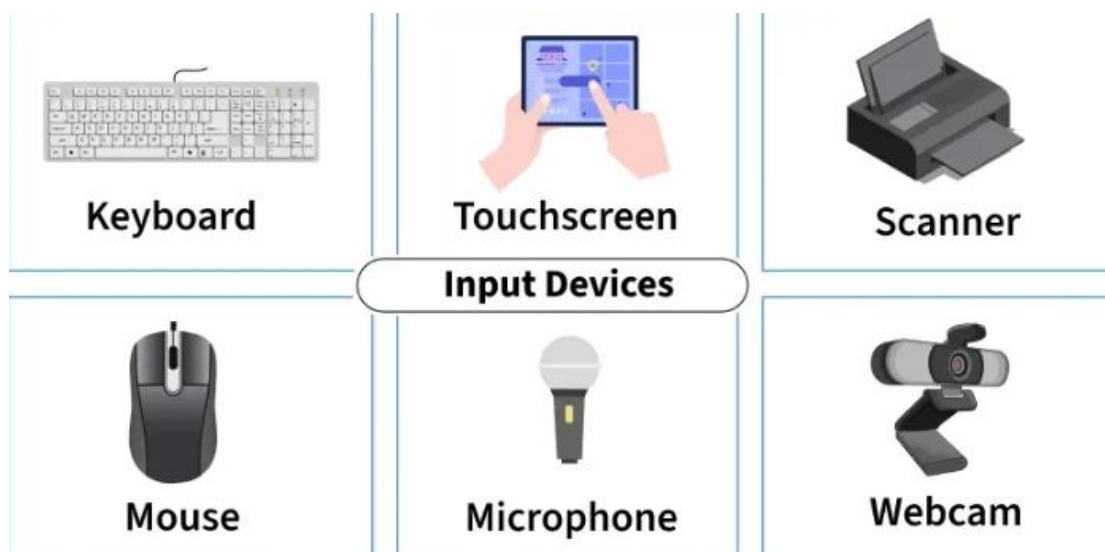
Computer Hardware Parts

These hardware components are further divided into the following categories, which are:

- **Input Devices**
- **Output Devices**
- **Storage Devices**
- **Hardware Components**

Input Devices

Input devices allow users to interact with a computer by entering data or commands. These devices convert the input into a format that the computer can process.

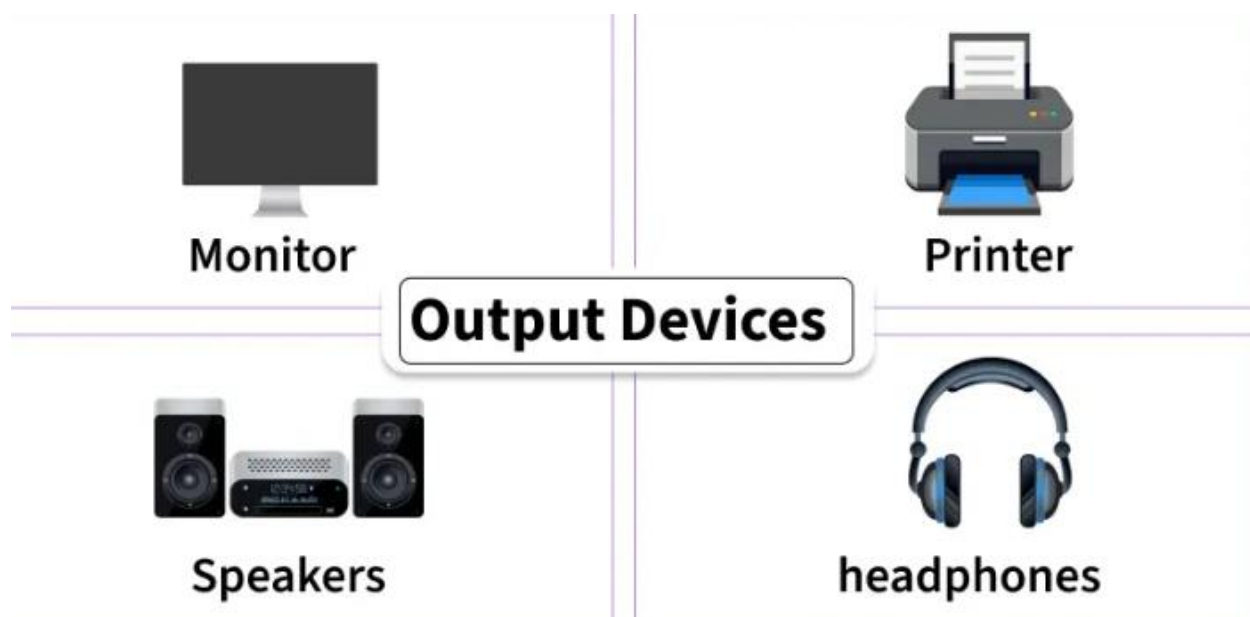


- **Keyboard:** The most widely used input device, featuring 104 keys, including alphabetic, numeric, and function keys. Modern keyboards connect via Bluetooth, replacing traditional wired connections.
- **Mouse:** A pointing device that controls the cursor on the screen. It features left, right, and middle buttons for selection and interaction. The sensor inside the mouse detects its movement speed, adjusting the cursor accordingly.
- **Scanner:** Scans documents, images, and other media, converting them into digital formats for editing or processing, similar to a Xerox machine.
- **Trackball:** A stationary pointing device with a ball that the user rotates to control the cursor, requiring less space than a traditional mouse.
- **Light Pen:** A light-sensitive pen used to draw or select objects on a CRT screen by detecting raster patterns, offering a direct interaction with the display.

- **Microphone:** Converts sound into electrical signals. It captures voice input for speech recognition and voice commands on the computer.
- **Optical Character Reader (OCR):** Scans printed or handwritten text, converting it into digital data by detecting reflected light from the characters, similar to a scanner.
- **Bar Code Reader:** Reads bar codes and converts them into digital data for processing. The bar code consists of light and dark lines that encode information.

Output Devices

Output devices display the results of tasks given to the computer in a human-readable form. Let's discuss some common output devices:



- **Monitor:** The main output device. It is also called VDU (visual display unit) and it looks like a TV screen. The Monitor displays the information from the computer. It is used to display text, video, images, etc.
- **Printer:** A printer is an output device that transfers data from the computer in a printed format by using text or images on paper. There are both coloured and black & white printers. Further, there are also different types of printers, like Laser Printer, Dot-matrix printers, and Inkjet printers.
- **Plotter:** It is similar to a printer but plotters are large. A plotter is used to generate large drawings, architectural blueprints, etc. on paper and these are high-quality images and drawings and large.
- **Speakers:** It is a very common output device and it gives sound as an output. The speaker is generally used to play music or anything having sound.

Storage Devices

Some devices are used for storage purposes and are known as secondary storage devices. Some of them are discussed below:

1. CD (Compact disc):

A CD is circular and made up of thin plated glass and plastic polycarbonate material. It has a storage capacity of 600 MB to 700 MB of data. It has a standard size of 12 cm with a hole in the centre of about 1.5 cm and 1.2 mm in thickness. There are 3 types of CDs, which are:

- **CD-ROM (CD - Read Only Memory):** Contents of this type of CD cannot be erased by the user. Only the publisher is allowed to access the data imprinted on this CD. CD-ROM is used for commercial purposes like for a music album or any application package by a software company.
- **CD-R (CD-Recordable):** In this, content or data can be stored once. After that, they can be read many times but the data or content cannot be rewritten or erased. (Kind of one-time use)
- **CD-RW (CD-Rewritable):** As the name suggests, this type of CD is used to rewrite the content or erase previous content and again write new content many times.

2. DVD (Digital Video/Versatile Disc):

A DVD is the same as a CD but with some more features. A DVD comes in single and dual-layer formats. It has much greater storage capacity in comparison to CD. The storage capacity of a DVD with a one-sided single layer is - 4.7 GB, one-sided double layer - 8.5 GB, double-sided single layer - 9.4 GB, and double-sided double layer - 17 GB. There are also some types of DVDs, which are :

- **DVD-ROM:** In this type, the contents of the DVD cannot be written on or erased by the user. DVD ROM is used for applications and databases for distributing them in large amounts.
- **DVD-R / DVD+R:** DVD-R (DVD minus R) and DVD+R (DVD plus R) are two different kinds of discs and they are once recordable formatst. Also, they have no difference virtually.
- **DVD-RW / DVD+RW:** This is a kind of rewritable disc and it allows up to 1,000 rewrites.
- **DVD-RAM:** DVD RAM is accessed like a hard disk. It provides high data security and storage capacity. This is a kind of rewritable disc and it allows up to 1,00,000 rewrites.

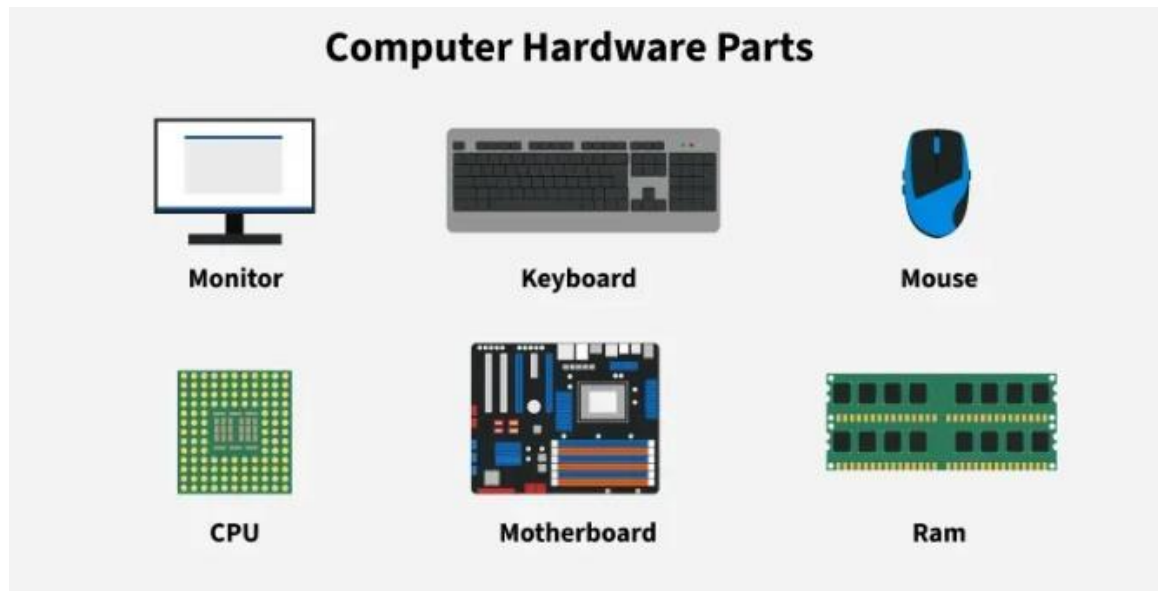
3. Hard Disk:

A hard disk is a non-volatile storage device that uses its read/write heads to store digital data on the magnetic surface of a rigid plate. It is generally 3.5 inches in size for desktops and 2.5 inches in size for laptops. A hard disk can be classified further into 3 types, which are:

- **Internal Hard Disk:** It has a common storage capacity stated as GB or TB. A system case or cabinet is the place where it is located. It can perform faster operations and its storage is fixed. It is mainly used to store large data files and programs.
- **Internal Cartridges:** The Internal hard disk can't be removed from the system cabinet easily. To resolve this problem Internal Cartridges are introduced. So, Internal cartridges make it easy to remove CDs. It has a storage capacity of 2 GB to 160 GB. It is used as an alternative to an internal hard disk.
- **Hard Disk Packs** are used by organizations such as banks and government sector organizations to store large amounts of data. They have a storage capacity of PB (peta bytes).

Hardware Components

Some important hardware devices known as the internal components are discussed below:



1. CPU (Central Processing Unit)

The CPU is also known as the heart of the computer. It consists of three units, generally known as the control unit, the Arithmetic Logical Unit (ALU), and the memory unit. Below is the block diagram of the CPU is given:

As shown in the diagram input is given to the CPU through input devices. This input goes to memory and the control unit gets instructions from memory. The control unit now decides what to do with the input or instructions and transfers it to ALU. Now, ALU performs various operations like addition, subtraction, multiplication, division, logical operations, etc. After that, the final result gets stored in memory and finally passed to output devices to give the output. So, this is how the CPU works.

2. Motherboard

It is the main circuit board inside a computer and it contains most of the electronic components together. All the components of the computer are directly or indirectly connected to the motherboard. It includes RAM slots, controllers, system chipsets, etc.

3. RAM (Random Access Memory)

It is also known as temporary or volatile memory. It holds the program and data, which are currently in process or processing. All the data is erased as soon as the computer is turned off or in case of a power failure. Data stored in this memory can be changed. There are two types of RAM:-

1. **SRAM (Static RAM):** SRAM consists of a flip-flop using a transistor or Mosfet (MOS). It is fast and has less access time. In this refreshing circuits are not required. But it is costly and requires more space. E.g. cache memory.
2. **DRAM (Dynamic RAM):** DRAM consists of capacitors and the data is stored in the form of capacitors. Capacitors charge when data is 1 and don't charge if data is 0. It requires refreshing circuits, as leakage of current in the capacitor can occur, so they need to be refreshed to the data. It is slower and has a higher access time. It is cheaper in comparison with SRAM. E.g. Main memory.

4. Video Graphics Array Port

A video input commonly used on computer monitors is called a video graphics array (VGA) port. Verifying that there isn't a loose connection, a damaged cable, or a broken display is one step in troubleshooting a VGA port. Compressed air can also be sprayed inside the VGA port by a computer expert to make sure it's dust-free.

5. Power Supply

All of a computer system's parts are powered by a power source. Typically, a power cord is used to connect a computer tower to an electrical outlet. By turning off the computer, unplugging and separating the power supply cord, or trying a different cord or socket, a technician can diagnose the power supply.

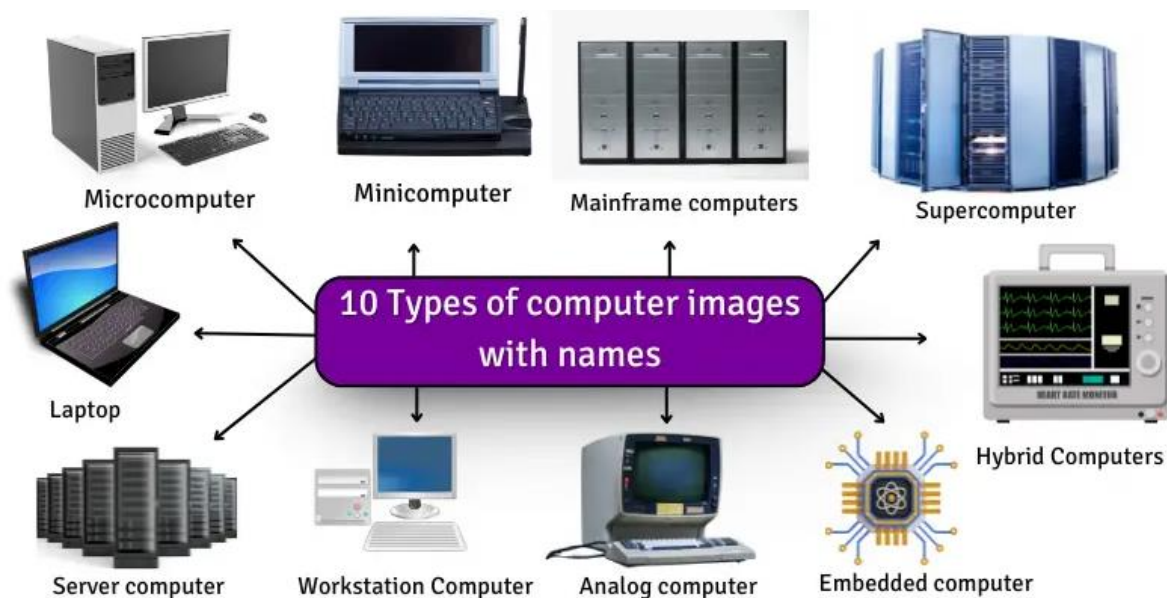
6. Cooling Fan

A computer's system to prevent overheating uses cooling fans. To aid customers who use their computers intensively, such as when streaming video or playing games, many computers contain more than one cooling fan. If a user detects their computer overheating, a computer expert might need to repair the cooling fan. The blades may be examined for any damage and cleared of any foreign objects. A technician's standard method of troubleshooting may involve replacing computer fans.

7. Hard Drive

On a computer system, files, programs, and other types of information are stored on hard drives, which are data storage devices. They utilise hard drives, which are magnetically coated discs used to store digital versions of information. A computer technician can suspect a corrupt hard disk when a hard drive dies.

Types of Computers:



Super computer

When we talk about speed, then the first name that comes to mind when thinking of computers is supercomputers. They are the biggest and fastest computers (in terms of speed of processing data). Supercomputers are designed such that they can process a huge amount of data, like processing trillions of instructions or data just in a second. This is because of the thousands of interconnected processors in supercomputers. It is basically used in scientific and engineering applications such as weather forecasting, scientific simulations, and nuclear energy research. It was first developed by Roger Cray in 1976.

Characteristics of Supercomputers

- Supercomputers are the computers that are the fastest and they are also very expensive.

- It can calculate up to ten trillion individual calculations per second, this is also the reason which makes it even faster.
- It is used in the stock market or big organizations for managing the online currency world such as Bitcoin etc.
- It is used in scientific research areas for analyzing data obtained from exploring the solar system, satellites, etc.

Main frame computer

Mainframe computers are designed in such a way that they can support hundreds or thousands of users at the same time. It also supports multiple programs simultaneously. So, they can execute different processes simultaneously. All these features make the mainframe computer ideal for big organizations like banking, telecom sectors, etc., which process a high volume of data in general.

Characteristics of Mainframe Computers

- It is also an expensive or costly computer.
- It has high storage capacity and great performance.
- It can process a huge amount of data (like data involved in the banking sector) very quickly.
- It runs smoothly for a long time and has a long life.

Minicomputer

Minicomputer is a medium size multiprocessing computer. In this type of computer, there are two or more processors, and it supports 4 to 200 users at one time. Minicomputer is similar to Microcontroller. Minicomputers are used in places like institutes or departments for different work like billing, accounting, inventory management, etc. It is smaller than a mainframe computer but larger in comparison to the microcomputer.

Characteristics of Minicomputer

- Its weight is low.
- Because of its low weight, it is easy to carry anywhere.
- less expensive than a mainframe computer.
- It is fast.

Workstation Computer

A workstation computer is designed for technical or scientific applications. It consists of a fast microprocessor, with a large amount of RAM and a high-speed graphic adapter. It is a single-user computer. It is generally used to perform a specific task with great accuracy.

Characteristics of Workstation Computer

- It is expensive or high in cost.
- They are exclusively made for complex work purposes.
- It provides large storage capacity, better graphics, and a more powerful CPU when compared to a PC.
- It is also used to handle animation, data analysis, CAD, audio and video creation, and editing.

Personal Computer (PC)

Personal Computers is also known as a microcomputer. It is basically a general-purpose computer designed for individual use. It consists of a microprocessor as a central processing unit(CPU), memory, input unit, and output unit. This kind of computer is suitable for personal work such as making an assignment, watching a movie, or at the office for office work, etc. For example, Laptops and desktop computers.

Characteristics of Personal Computer (PC)

- In this limited number of software can be used.
- It is the smallest in size.
- It is designed for personal use.
- It is easy to use.

Server Computer

Server Computers are computers that are combined data and programs. Electronic data and applications are stored and shared in the server computer. The working of a server computer is that it does not solve a bigger problem like a supercomputer but it solves many smaller similar ones. Examples of server computer are like Wikipedia, as when users put a request for any page, it finds what the user is looking for and sends it to the user.

Characteristics:

- Handles **data and applications** for multiple clients or users.
- **Centralized management** of resources (e.g., databases, websites).
- Designed to run **24/7** without interruption.
- More powerful than a typical PC and optimized for **networking and data management**.

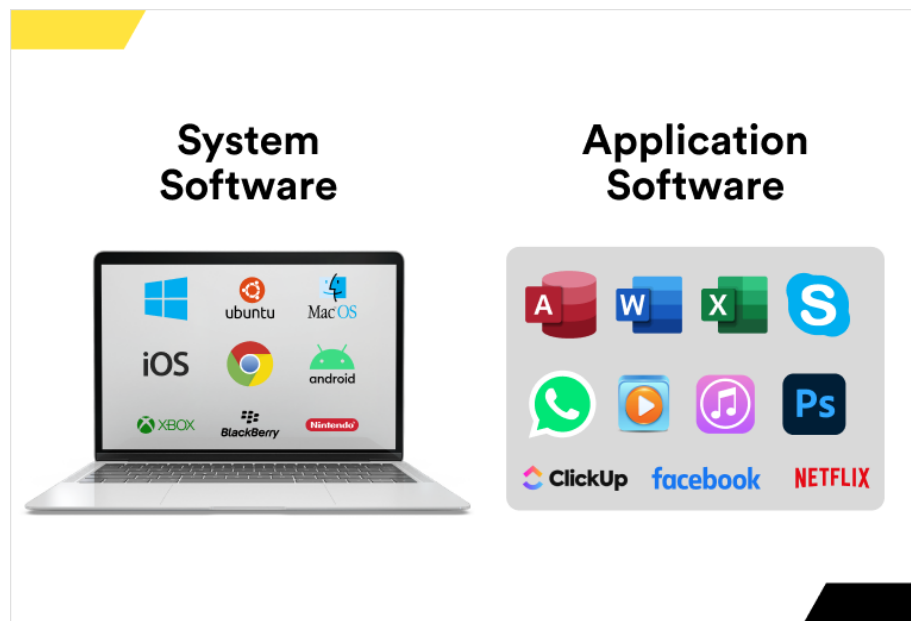
Examples:

- **Web servers** (e.g., Apache HTTP Server)
- **Database servers** (e.g., SQL Server, Oracle)

Software: System software and Application software.

- In the era of Digitalization and Modernization, Software is the very crucial support that allows hardware to perform various useful tasks. There are two categories of software; System Software and Application Software, these types perform different work which is why it is crucial to understand the difference.
- Application software is created to help users to perform specific tasks directly and System software acts as a mediator between hardware and user applications. Computer Software is a sort of program that allows clients to work on different assignments or use them to work on their System. It tells the working and responsibilities of the System.
- Basically, Software is a set of instructions or commands that tells a user how to do and what to do. In this article, we will look into these topics in detail along with their differences.

Types of Software



- System Software
- Application Software

What is System Software?

System Software is the type of software that is the interface between application software and the system. Low-level languages are used to write the system software. System Software maintains the system resources and gives the path for application software to run. An important thing is that without system software, the system can not run. It is general-purpose software.

Functions of System Software

- Memory Management
- Processor Management
- File Management
- Security
- Error-detecting Aids
- Scheduling

Features of System Software

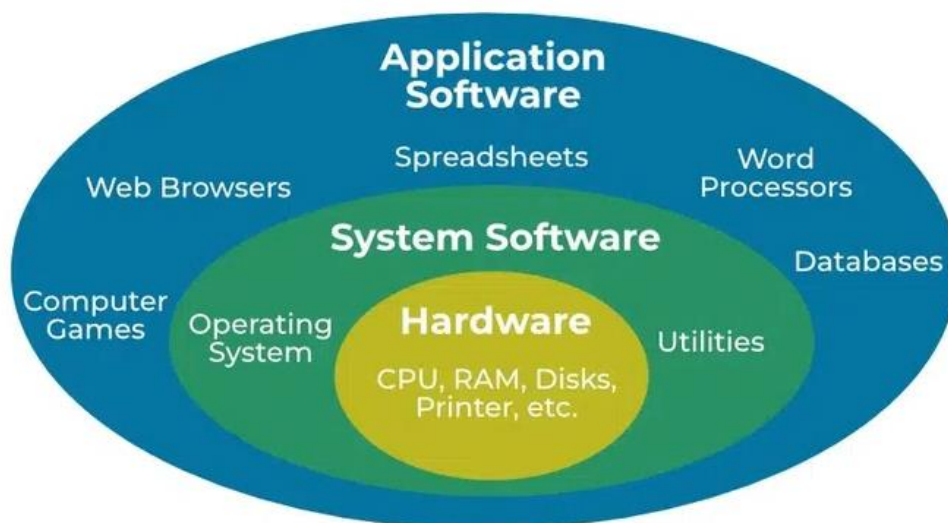
- System software is written in a low-level language.
- The size of the system Software is smaller.
- System software is complex to understand.
- System software is present near hardware components.

Types of System Software

1. Operating System: Operating System is the main part of the Computer System. It has the responsibility of managing all the resources such as CPU, Printer, Hard Disk, etc. It also provides services to many other Computers Softwares. Examples of Operating Systems are Linux, Apple, macOS, Microsoft Windows, etc.

2. Language Processor: System Software converts Human-Readable Language into a Machine Language and it is done by Language Processor. It converts programs into instructions that are easily readable by Machines.

3. Device Driver: A Device Driver is a program or software that helps to perform its functions by controlling the device. You first have to install a driver for running the program.



What is Application Software?

Application Software is the type of software that runs as per user request. It runs on the platform which is provided by system software. High-level languages are used to write the application software. It's specific purpose software. The main difference between System Software and Application Software is that without system software, the system cannot run on the other hand without application software, the Low-level maintenance system always runs.

Functions of Application Software

- Information and data management
- Management of documents (document exchange systems)
- Development of visuals and video
- Emails, text messaging, audio, and video conferencing, and cooperation are all options.
- Management of accounting, finance, and payroll
- Management of resources (ERP and CRM systems)

Features of Application Software

- Application software is written in a high-level language.
- Application software requires more storage space than system software.
- Only a single task is performed by each application software.
- Application Software is easy to build in comparison to system software.

Types of Application Software

1. General Purpose Software: This Application Software is used to perform tasks that are used for a variety of tasks, just not limited to a specific task only. For Example, MS Word, MS Excel, etc.

2. Customized Software: It is used to perform tasks that are designed for specific organizations. For Example, Railway Reservation System, Airline Reservation System, etc.

3. Utility Software: It is used to support the architecture of the Computer. It is designed for optimizing and maintaining the system and also taking care of its requirements.

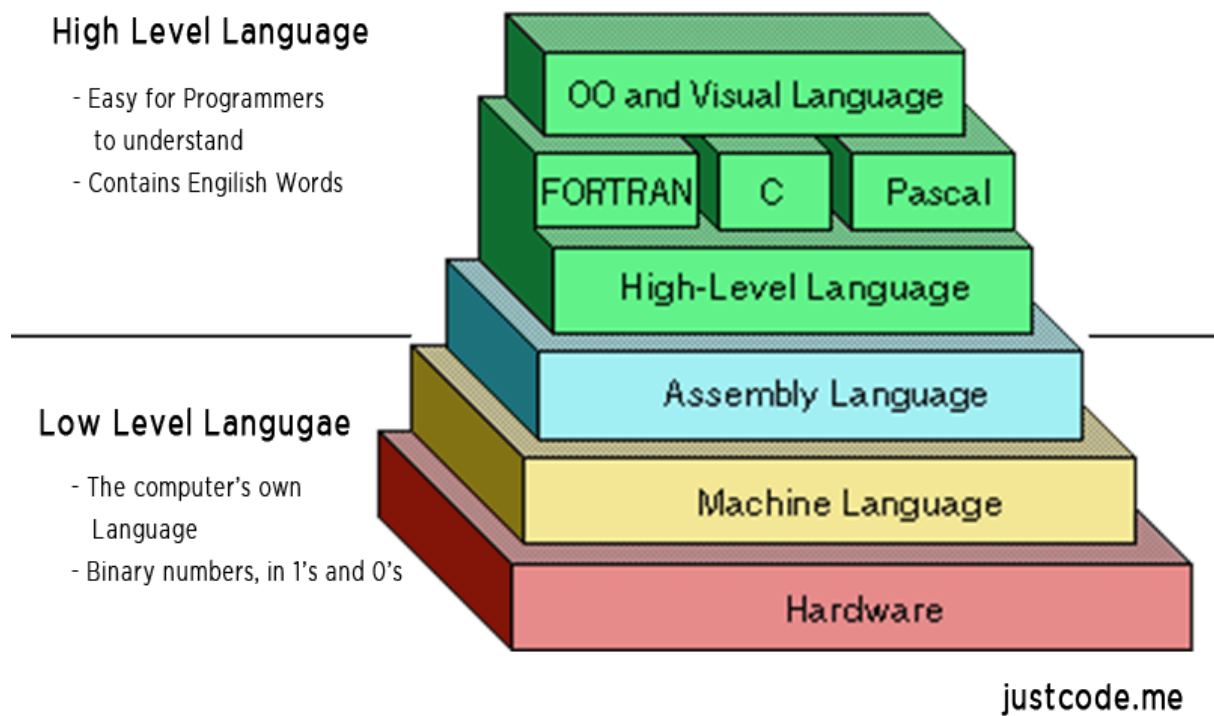
Difference between System Software and Application Software

System Software	Application Software
System Software maintains the system resources and gives the path for application software to run.	Application software is built for specific tasks.
Low-level languages are used to write	While high-level languages are used to

the system software.	write the application software.
It is general-purpose software.	While it's a specific purpose software.
Without system software, the system stops and can't run.	While Without application software system always runs.
System software runs when the system is turned on and stops when the system is turned off.	While application software runs as per the user's request.
Example: System software is an operating system, etc.	Example: Application software is Photoshop, VLC player, etc.
System Software programming is more complex than application software.	Application software programming is simpler in comparison to system software.
The Software that is designed to control, integrate and manage the individual hardware components and application software is known as system software.	A set of computer programs installed in the user's system and designed to perform a specific task is known as application software.
A system software operates the system in the background until the shutdown of the computer.	Application software runs in the front end according to the user's request.
The system software has no interaction with users. It serves as an interface between hardware and the end user.	Application software connects an intermediary between the user and the computer.
System software runs independently.	Application software is dependent on system software because they need a set platform for its functioning.

Programming Languages:

Programming languages are the foundation of software development, allowing developers to create applications, websites, and systems through computer-understandable instructions.



- They use a formal syntax and rules to write code that is translated into machine language for execution by the computer.
- Programming languages bridge human logic and machine execution, providing a structured way to solve problems.
- Different languages serve different purposes — for example, Python is beginner-friendly and versatile, while C++ is used for high-performance tasks like game development.

Types of Programming Languages

Programming languages are classified based on their level of abstraction:

- **High-Level Languages:** Human-readable and are portable across computers. Examples: Python, Java, JavaScript. They need a compiler or interpreter to run.
- **Low-Level Languages:** Close to machine code, giving more control over hardware but harder to write. Examples: Machine code.
- **Assembly Languages:** A mix between high- and low-level, using simple symbolic instructions for a computer. An assembler converts it into machine code.

Key Terms in Programming Languages

- **Syntax:** Rules for writing code, like grammar in human languages (e.g., C uses {}, Python uses indentation).

- **Data Types:** Types of data a program can handle, like integers, floats, strings, and booleans.
- **Variables:** Named storage for data that can change during program execution (e.g., score = 100).
- **Operators:** Symbols like +, -, *, /, and == for arithmetic or comparisons.
- **Control Statements:** Manage program flow using if-else, loops, and function calls.
- **Libraries & Frameworks:** Pre-written code to simplify tasks (e.g., Python's pandas, JavaScript's React).
- **Paradigms:** Programming styles, like Procedural (C), Object-Oriented (Java), and Functional (Haskell).

Popular Programming Languages

Here are some widely used programming languages and their typical applications:

- **Python:** Known for simplicity and readability, used in data science, machine learning, and web development.
- **Java:** Platform-independent, popular for enterprise applications and Android development.
- **JavaScript:** Essential for web development, enabling interactive websites.
- **C++:** High-performance language for system programming, games, and real-time applications.
- **C#:** Used for Windows applications and game development with Unity.
- **Swift:** Apple's language for iOS and macOS apps.
- **Go:** Designed for scalability and cloud computing.
- **Ruby:** Known for web frameworks like Ruby on Rails.

Machine language

- Machine language is a low-level programming language that is understood by computers. Machine language is made up of binary bits 0 and 1.
- Machine language is also known as machine codes or object code. As machine language consists of only 0 and 1, that's why it is difficult to understand in raw form.
- Machine language cannot understand by humans. The CPU processes this machine code as input. In this article, we are going to learn about what is Machine language, the features of machine language, the advantages and disadvantages of machine learning, and why it is difficult for humans to understand machine language(low-level language).

What is Machine Language?

Machine language is a low-level programming language that consists of binary bits i.e. only 0 and 1. The data present in binary form is the reason for its fast execution. In Machine language, instructions are directly executed by the CPU. Machine language is also known as object code or machine code. Machine language is binary language.

```
01001010101
10110010101
10001110100
110010101011
110101010101
```

Machine Language

Needs of Machine Language

As a human, we write code in high level language. The programming language which we use to write codes such as C, C++ and java are high level languages. High level language is not understood by computer directly so it is converted into low level machine language to understand the meaning of code and perform execution. Computers compile the code written by us and translate into machine code and then execute it. Computers are only able to understand machine language.

Features of Machine Language

Below is some feature of Machine Language.

- Machine language is a low level language.
- Machine language consists of only 0 and 1 bits.
- Machine languages are platform dependent.
- It is nearly impossible to learn machine language for humans because it requires a lot of memorization.
- Machine language is used to create and construct drivers as well.

Understand the Complexity of Machine language

In machine language every character, integer and special symbols are written in form of 0 and 1. To understand machine language let's take an example of a machine language instruction. This is a simple addition operation: 01100110 00001010. This binary sequence represents an instruction that tells the computer to add two numbers together.

Meaning of Binary bits in Machine Language:

A sequence of bits is used to give commands in machine languages.

- The 1s (one) represents the true or on states.
- On the other hand, the 0s (zero) represent the off or false states.
- That's why no human can remember the binary codes of machine languages. As a result, learning these languages is not possible for humans.

Machine Language Instruction Components

Machine language consist of two instruction components :

1. Operand(s)

The operand(s) represents the data that the operation must be performed on. This data can take various forms, depending on the processor's architecture. This can be a register containing a value, a memory address pointing to a location in memory where the data is stored, or a constant value embedded within the instruction itself.

2. Opcode

The opcode (Operation code) represents the operation that the processor must perform. This indicates that the instruction is an arithmetic operation such as addition, subtraction, multiplication, or division.

Advantages of Machine Language

Some advantages of machine language are listed below:

- Machine languages are faster in execution because they are in binary form.
- Machine language does not need to be translated, because it is already present in simple binary form.
- The CPU directly executes the machine language.
- The evolution of the computer system and operating system over the time period is due to machine language.
- Machine languages are used in developing a high-grade computer system.

Disadvantages of Machine Language

Some disadvantages of machine language are listed below:

- Machine language are complex to understand and memorize.
- Writing codes in machine language is time-consuming.

- It is very difficult to resolve bugs and errors present in the codes and programs.
- Codes written in machine languages are more prone to error.
- Machine languages are not easy to modify.
- Machine language are platform Independent.

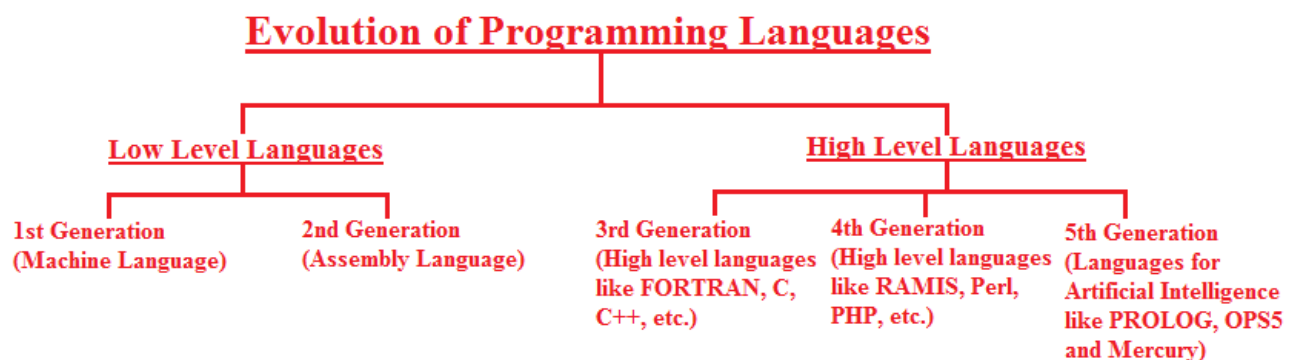
Assembly language,

What is Assembly Language?

Assembly language is a low-level language that helps to communicate directly with computer hardware. It uses mnemonics to represent the operations that a processor has to do. Which is an intermediate language between high-level languages like C++ and the binary language. It uses hexadecimal and binary values, and it is readable by humans.

Evolution of Assembly Language?

Assembly language has evolved hand in hand with advancements in computer hardware and the evolving needs of programmers. Here's a closer look at each generation:



First Generation (1940-1950):

- Computers relied on vacuum tubes, and programming was directly in machine language, using binary instructions.
- Assembly language emerged as a readable abstraction, employing mnemonic codes to represent machine instructions.

Second Generation (1950-1960):

- Transistor-based computers replaced vacuum tubes, offering enhanced Consistency and prowess.
- Assembly languages became more intricate to handle the complex instruction sets of these new machines. Simultaneously, high-level programming languages like FORTRAN and COBOL provided Advanced abstraction

Third Generation (1960-1970):

- Integrated circuits became Standard place, resulting in Diminished but potent computers.
- Assembly languages evolved further, introducing features like macros and symbolic labels, which boosted programmer productivity and code readability.

Fourth Generation (1970-1980):

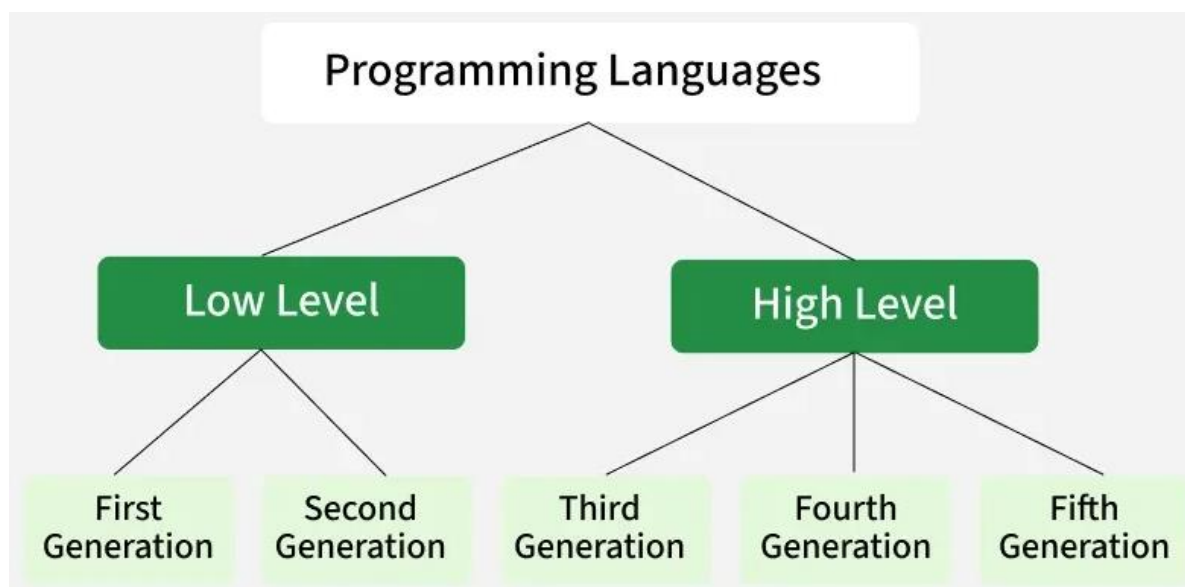
- The beginning of microprocessors transformed computing, paving the way for microcomputer systems such as the IBM PC and Apple II.
- Assembly languages for microcomputers were redesigned to enhance user accessibility, featuring syntax highlighting and automatic indentation, thus increasing inclusivity for a larger group of programmers.

Fifth Generation (1980-present):

- This era is characterized by executing multiple computational tasks simultaneously this method is known as parallel processing system and the growth of sophisticated software systems
- Assembly language continued to evolve to meet the demands of programmers, with the deployment of cutting-edge debugging methods and tools focused on improving code performance and productivity for intricate systems.

High-level language, 4GL and 5GL

Programming languages have evolved significantly over time, moving from fundamental machine-specific code to complex languages that are simpler to write and understand. There are five generations of Programming languages. They are:



1. First-Generation Language:

The first-generation languages are also called machine languages/ 1G language. This language is machine-dependent. The machine language statements are written in binary code (0/1 form) because the computer can understand only binary language.

Advantages:

1. Fast & efficient as statements are directly written in binary language.
2. No translator is required.

Disadvantages:

1. Difficult to learn binary codes.
2. Difficult to understand - both programs & where the error occurred.

2. Second Generation Language:

The second-generation languages are also called assembler languages/ 2G languages. Assembly language contains human-readable notations that can be further converted to machine language using an assembler.

Assembler - converts assembly level instructions to machine-level instructions.

Programmers can write the code using symbolic instruction codes that are meaningful abbreviations of mnemonics. It is also known as low-level language.

Advantages:

1. It is easier to understand if compared to machine language.
2. Modifications are easy.
3. Correction & location of errors are easy.

Disadvantages:

1. Assembler is required.
2. This language is architecture /machine-dependent, with a different instruction set for different machines.

3. Third-Generation Language:

The third generation is also called procedural language /3 GL. It consists of the use of a series of English-like words that humans can understand easily, to write instructions. It's also called High-Level Programming Language. For execution, a program in this language needs to be translated into machine language using a Compiler/ Interpreter. Examples of this type of language are C, PASCAL, FORTRAN, COBOL, etc.

Advantages :

1. Use of English-like words makes it a human-understandable language.
2. Lesser number of lines of code as compared to the above 2 languages.

3. Same code can be copied to another machine & executed on that machine by using compiler-specific to that machine.

Disadvantages:

1. Compiler/ interpreter is needed.
2. Different compilers are needed for different machines.

4. Fourth Generation Language:

The fourth-generation language is also called a non - procedural language/ 4GL. It enables users to access the database. Examples: SQL, Foxpro, Focus, etc.

These languages are also human-friendly to understand.

Advantages:

1. Easy to understand & learn.
2. Less time is required for application creation.
3. It is less prone to errors.

Disadvantages:

1. Memory consumption is high.
2. Has poor control over Hardware.
3. Less flexible.

5. Fifth Generation Language:

The fifth-generation languages are also called 5GL. It is based on the concept of artificial intelligence. It uses the concept that rather than solving a problem algorithmically, an application can be built to solve it based on some constraints, i.e., we make computers learn to solve any problem. Parallel Processing & superconductors are used for this type of language to make real artificial intelligence.

Examples: PROLOG, LISP, etc.

Advantages:

1. Machines can make decisions.
2. Programmer effort reduces to solve a problem.
3. Easier than 3GL or 4GL to learn and use.

Disadvantages:

1. Complex and long code.
2. More resources are required & they are expensive too.

Features of good programming language.

- **Programming language** is a set of instructions and rules that allow humans to communicate with computers.

- A **good programming language** helps programmers develop efficient, reliable, and easy-to-understand programs.
- Every programming language has its own syntax and structure, but the best languages share common features that make program development easier, faster, and less error-prone.

Features of a Good Programming Language

1. Simplicity

- A good programming language should be **simple, clear, and easy to learn**.
- The syntax should be easy to understand and use without unnecessary complexity.
- Example: Python is simple because its syntax is close to natural English.

2. Readability

- The language should allow programs to be **easily read and understood** by others.
- Good indentation, naming conventions, and meaningful keywords improve readability.
- Example: A readable program is easy to debug and maintain.

3. Portability

- A good language should be **machine-independent**, meaning programs written in it can run on different hardware systems with minimal modification.
- Example: Java is highly portable due to its *“Write Once, Run Anywhere”* concept.

4. Efficiency

- The language should enable programs to **run quickly and use memory efficiently**.
- It should provide features for optimizing performance, like memory management or direct hardware access.

5. Reliability

- Programs written in the language should perform **consistently without errors or crashes**.
- A good programming language should include strong error-handling and debugging features.

6. Flexibility

- A good programming language should allow both **small and large-scale program development**.
- It should support multiple programming paradigms such as *procedural, object-oriented, and functional programming*.

7. Extensibility

- The language should allow the addition of **new features or libraries** without changing the core structure.
- Example: Python and C++ allow users to add modules and libraries to extend functionality.

8. Security

- The language should protect data and restrict unauthorized access.
- It should have features like **type checking, exception handling, and memory protection**.
- Example: Java provides built-in security through the Java Virtual Machine (JVM).

9. Support for Modularity

- A good language should support **modular programming**, i.e., dividing a large program into smaller, manageable modules or functions.
- This improves **code reuse** and **maintenance**.

10. Comprehensive Library Support

- A rich set of **standard libraries** and built-in functions reduces development time.
- Example: Python, Java, and C# have extensive standard libraries for mathematical, graphical, and database operations.

11. Maintainability

- A good programming language allows easy **modification and updating** of existing code.
- Well-structured and documented programs are easier to maintain.

12. Robust Error Handling

- It should help detect, report, and recover from errors during execution.
- Exception handling mechanisms make programs more reliable and user-friendly.

13. Generality

- The language should be **general-purpose**, supporting various application areas like web, mobile, and system programming.
- A **good programming language** is one that is **simple, efficient, portable, readable, secure, and easy to maintain**.
- It helps programmers write code that is reliable, reusable, and adaptable to new requirements.

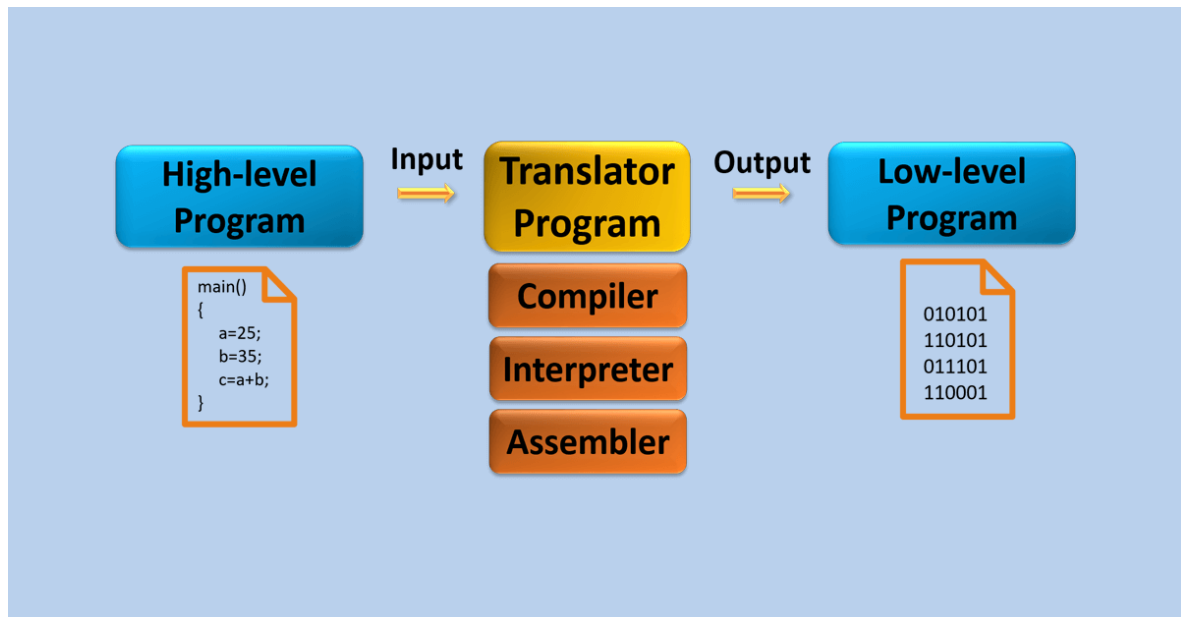
- Languages like **Python, Java, and C++** are considered good programming languages because they exhibit most of these features.

Summary Table

Feature	Description
Simplicity	Easy to learn and understand
Readability	Clear and structured code
Portability	Works on multiple systems
Efficiency	Fast execution and optimal memory use
Reliability	Consistent, error-free performance
Flexibility	Suitable for various applications
Extensibility	Easy to add new features
Security	Prevents unauthorized access
Modularity	Allows code division into modules
Library Support	Built-in functions and tools
Maintainability	Easy to modify and debug

Translators: Interpreters and Compilers.

- Compilers, interpreters, translate programs written in high-level languages into machine code that a computer understands and assemblers translate programs written in low-level or assembly language into machine code. In the compilation process, there are several stages. To help programmers write error-free code, tools are available.
- Assembly language is machine-dependent, yet mnemonics used to represent instructions in it are not directly understandable by machine and high-Level language is machine-independent.
- A computer understands instructions in machine code, i.e. in the form of 0s and 1s. It is a tedious task to write a computer program directly in machine code. The programs are written mostly in high-level languages like Java, C++, and Python etc. and are called source code.
- This source code cannot be executed directly by the computer and must be converted into machine language to be executed. Hence, a special translator system software is used to translate the program written in a high-level language into machine code is called Language Processor and the program after translated into machine code (object program/object code).



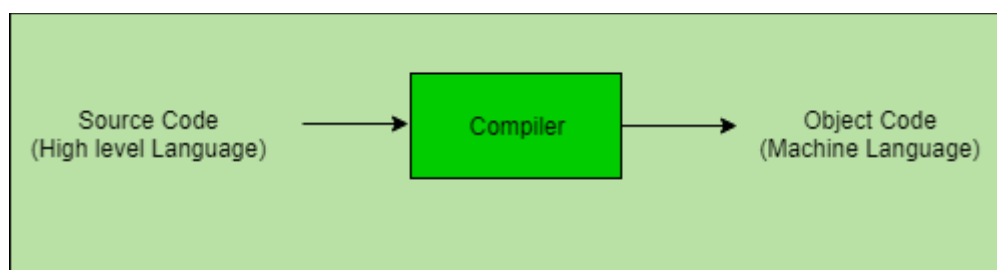
Types of Language Processors

The language processors can be any of the following three types:

1. Compiler

The language processor that reads the complete source program written in high-level language as a whole in one go and translates it into an equivalent program in machine language is called a Compiler. Example: C, C++, C#.

In a compiler, the source code is translated to object code successfully if it is free of errors. The compiler specifies the errors at the end of the compilation with line numbers when there are any errors in the source code. The errors must be removed before the compiler can successfully recompile the source code again the object program can be executed number of times without translating it again.

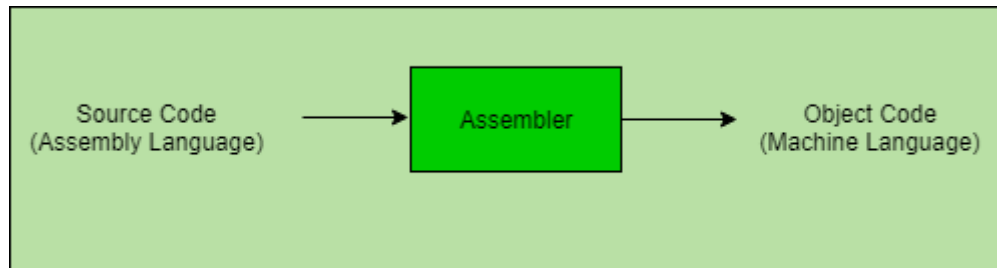


2. Assembler

The Assembler is used to translate the program written in Assembly language into machine code. The source program is an input of an assembler that contains assembly language instructions. The output generated by the assembler is the object code or machine code understandable by the computer. Assembler is basically the 1st interface that is able to communicate humans with the machine. We need an assembler to fill the gap between

human and machine so that they can communicate with each other. code written in assembly language is some sort of mnemonics(instructions) like ADD, MUL, MUX, SUB, DIV, MOV and so on. and the assembler is basically able to convert these mnemonics in binary code. Here, these mnemonics also depend upon the architecture of the machine.

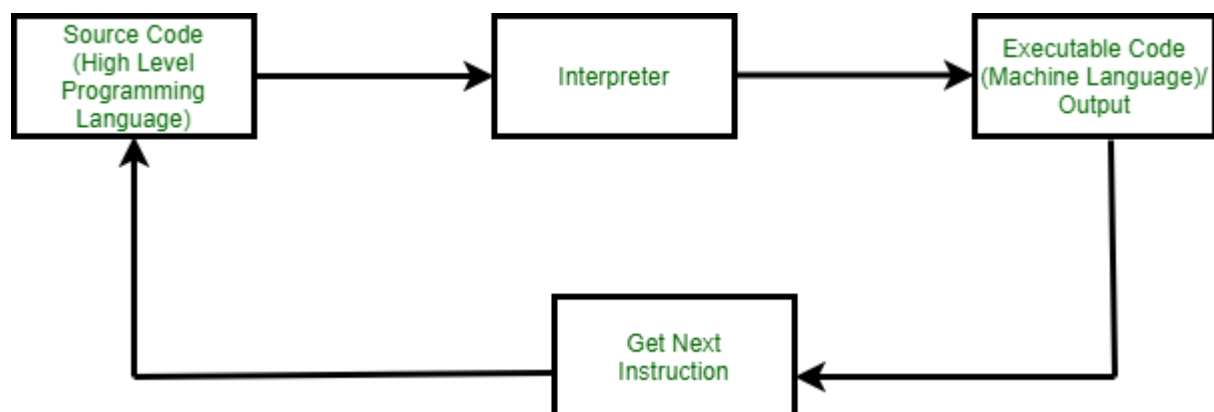
For example, the architecture of intel 8085 and intel 8086 are different.



3. Interpreter

The translation of a single statement of the source program into machine code is done by a language processor and executes immediately before moving on to the next line is called an interpreter. If there is an error in the statement, the interpreter terminates its translating process at that statement and displays an error message. The interpreter moves on to the next line for execution only after the removal of the error. An Interpreter directly executes instructions written in a programming or scripting language without previously converting them to an object code or machine code. An interpreter translates one line at a time and then executes it.

Example: Perl, Python and Matlab.



Difference between Compiler and Interpreter

Compiler	Interpreter
A compiler is a program that converts the entire source code of a programming	An interpreter takes a source program and runs it line by line, translating each line as

Compiler	Interpreter
language into executable machine code for a CPU.	it comes to it.
The compiler takes a large amount of time to analyze the entire source code but the overall execution time of the program is comparatively faster.	An interpreter takes less amount of time to analyze the source code but the overall execution time of the program is slower.
The compiler generates the error message only after scanning the whole program, so debugging is comparatively hard as the error can be present anywhere in the program.	Its Debugging is easier as it continues translating the program until the error is met.
The compiler requires a lot of memory for generating object codes.	It requires less memory than a compiler because no object code is generated.
Generates intermediate object code.	No intermediate object code is generated.
For Security purpose compiler is more useful.	The interpreter is a little vulnerable in case of security.
Examples: C, C++, C#	Examples: Python, Perl, JavaScript, Ruby.

Unit 1 Question Bank

2-Mark Questions

1. What is a computer?
 2. What are the main components of the CPU?
 3. Define system software and application software.
 4. What is machine language?
 5. Write any two features of a good programming language.
-

5-Mark Questions

1. Explain the characteristics and limitations of a computer.
 2. Write short notes on input and output devices with examples.
 3. Differentiate between system software and application software.
-

10-Mark Questions

1. Describe the **history and evolution of computers** in detail.
2. Explain the **characteristics and limitations of a computer** with examples.
3. Explain the **hardware components** of a computer (CPU, memory, storage, input/output devices).
4. Describe the **different types of computers** and their uses.
5. Explain the **classification of software** and discuss system software and application software with examples.
6. Describe the **types of programming languages** and explain machine, assembly, high-level, 4GL, and 5GL.
7. Explain in detail about **translators – compilers and interpreters** with differences.
8. Discuss the **features of a good programming language** with suitable examples.
9. Explain the **functional units of a computer system**.
10. Compare **all generations of programming languages** with examples.

UNIT II

Data

Data is a collection of raw facts, figures, and symbols that represent values but have no meaning on their own. Data can be in the form of numbers, characters, images, sounds, or any measurable values. When data is organized and processed, it becomes *information*, which is useful for decision-making.

Example:

- Raw data: 95, 88, 76
- Information: "Marks of a student in three subjects are 95, 88, and 76."

Data Types

Data types specify what kind of data can be stored in a variable and what operations can be performed on it. Choosing the correct data type ensures efficient memory usage and accurate results.

Common Data Types:

1. **Integer (int)** – Stores whole numbers (positive or negative).
Example: `int age = 23;`
2. **Float/Real (float, double)** – Stores decimal numbers (with fraction part).
Example: `float pi = 3.1416;`
3. **Character (char)** – Stores a single character like a letter, digit, or symbol.
Example: `char grade = 'A';`
4. **String (char[], string)** – Stores a sequence of characters (like words or sentences).
Example: `char name[] = "Anitha";`
5. **Boolean (bool)** – Stores only two values: true or false.
Example: `bool pass = true;`

Input

Definition:

Input is the process of providing data or instructions to a computer system for processing. Input can come from input devices (keyboard, mouse, scanner, microphone) or from files and networks.

Example in C:

```
int x;  
printf("Enter a number: ");
```

```
scanf("%d", &x); // user enters a number
```

Example in Python:

```
x = int(input("Enter a number: "))
```

Processing of Data

Definition:

Processing is the operation performed on input data by the CPU to convert it into meaningful output. This may involve calculations, comparisons, sorting, searching, or logical operations.

Example:

- Input: a = 10, b = 5
- Processing: sum = a + b;
- Output: 15

The CPU performs this processing using its **Arithmetic Logic Unit (ALU)** and **Control Unit (CU)**.

Arithmetic Operators

Arithmetic operators are symbols used to perform basic mathematical operations in a program.

Operators and Their Meaning:

- + → Addition
- - → Subtraction
- * → Multiplication
- / → Division (returns quotient)
- % → Modulus (returns remainder)

Examples:

- $7 + 3 = 10$
 - $7 - 3 = 4$
 - $7 * 3 = 21$
 - $7 / 3 = 2$ (integer division in C)
 - $7 \% 3 = 1$
-

Hierarchy of Operations

The hierarchy (or precedence) of operations defines the order in which mathematical operations are performed in an expression.

Order of Evaluation:

1. Parentheses ()
2. Exponentiation (where supported)
3. Multiplication *, Division /, Modulus %
4. Addition +, Subtraction -

Examples:

- Expression: $2 + 3 * 4$
 - First $3 * 4 = 12$
 - Then $2 + 12 = 14$
- Expression: $(2 + 3) * 4$
 - Parentheses first: $5 * 4 = 20$

Thus, parentheses can be used to change the normal priority of operations.

Output

Output is the result of data processing, presented to the user in a meaningful way. Output may be displayed on monitors, printed on paper, stored in a file, or sent to another device.

Example in C:

```
int sum = 15;
printf("The sum is %d", sum);
```

Output:

The sum is 15

Example in Python:

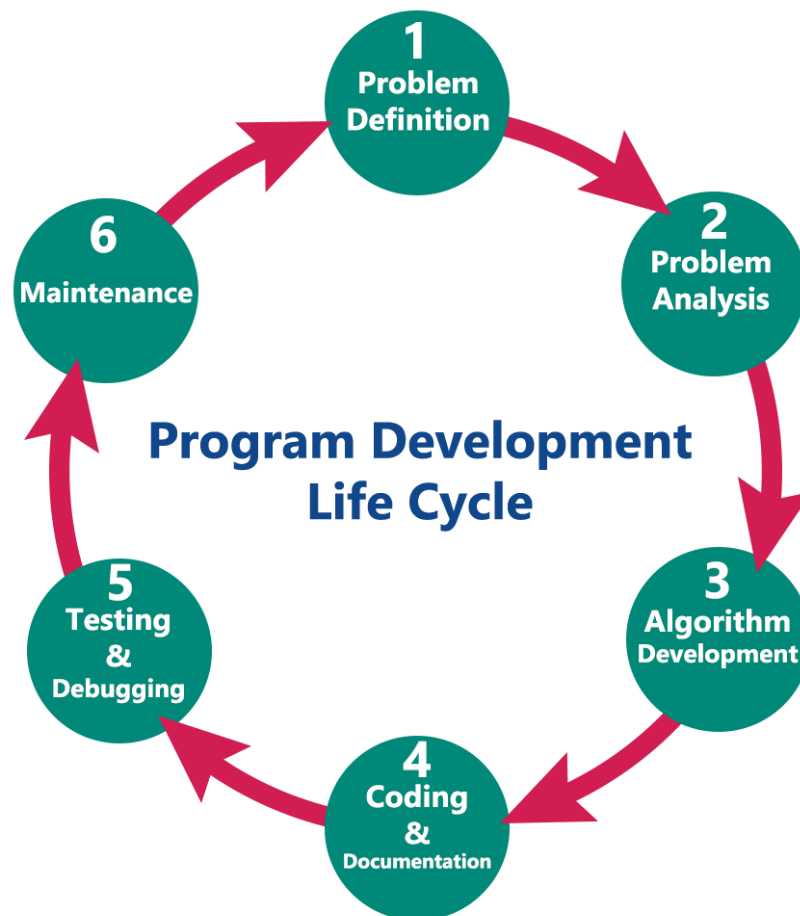
```
sum = 15
print("The sum is", sum)
```

Program Development Life Cycle (PDLC)

What is PDLC?

- The **Program Development Life Cycle (PDLC)** is a step-by-step process used to create computer programs.
- It helps programmers make sure the program works correctly, meets the needs of users, and is easy to maintain.

- PDLC is similar to **SDLC (Software Development Life Cycle)** but focuses more on **developing individual programs**.
- It is **iterative** (steps can be repeated with improvements).



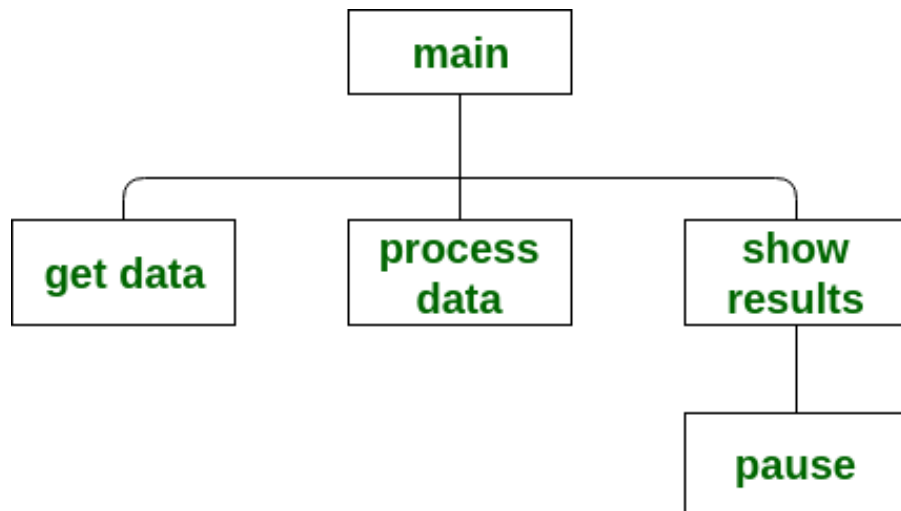
Steps in PDLC

1. Defining the Problem

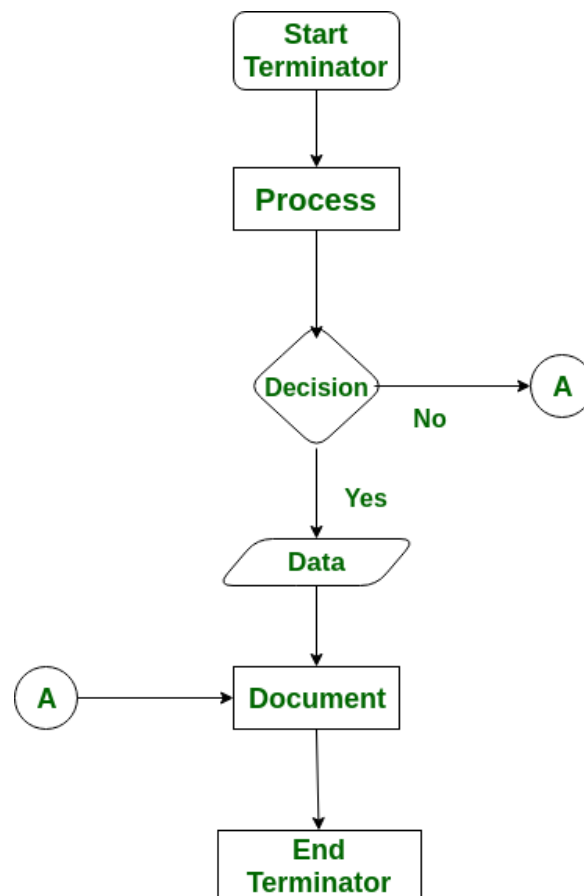
- Clearly describe what problem the program should solve.
- Example: A system analyst prepares a **program specification** (data, processing, output, and interface).

2. Designing the Program

- Break the program into smaller parts (modules).
- Use **modular programming** → main routine + sub-modules.
- Use program design tools like:
 - **Structure Charts (Hierarchy Charts):** Show tasks in a top-down way. The **top module** is the Main/Control module.



- **Algorithms:** An algorithm is a step-by-step description of how to arrive at a solution in the easiest way. Algorithms are not restricted to computer world only. In fact, we use them in everyday life.
- **Flowcharts:** A flowchart is a diagram that shows the logic of the program. For example:



- **Decision tables:** A Decision table is a special kind of table, which is divided into four parts by a pair of horizontal and vertical lines.

- **Pseudo code:** A pseudo code is another tool to describe the way to arrive at a solution. They are different from algorithm by the fact that they are expressed in program language like constructs.

3. Coding the Program

- Translating the **algorithm/flowchart/pseudo code** into a specific **programming language** (C, Java, Python, etc.).
- Uses **structured programming** (sequence, selection, iteration).
- Programmer must follow syntax rules of the language.
- **Errors in syntax** must be removed before execution.

4. Testing and Debugging the Program

- After fixing syntax errors, the program runs but may give **wrong output** due to **logical errors**.
- **Syntax Errors** → Wrong grammar/format (e.g., missing ;).
- **Logical Errors** → Wrong solution/logic (e.g., using + instead of *).
- Both errors are called **Bugs**.
- The process of finding and fixing bugs is called **Debugging**.
- **Test Data** is used to check correctness.

5. Documenting the Program

- Documentation helps other programmers, users, and testers.
- Includes:
 - Structure charts
 - Pseudo code
 - Flowcharts
 - Decision tables
- Final manual includes:
 - Program overview
 - Tutorials
 - Explanation of features
 - Error messages and solutions

6. Deploying and Maintaining the Program

- **Deployment** → Install program at user's site.
- Program is monitored until approved by the user.

- **Maintenance** → Continuous process of fixing errors and updating program to meet new requirements.

Benefits of PDLC

1. **Structured approach** → Logical, step-by-step development.
2. **Better communication** → Between developers, managers, and users.
3. **Risk management** → Identifies and controls risks early.
4. **Improved quality** → Systematic testing improves final product.
5. **Efficiency** → Saves time and resources in the long run.

Limitations of PDLC

1. **Time-consuming** → strict phases may delay development.
2. **Inflexible** → not easy to make changes once process starts.
3. **Costly** → requires more resources and budget.
4. **Complex** → needs trained staff and good planning.
5. **Not suitable for small projects** → too heavy for simple tasks.

Structured Programming

Structured programming is a method of writing programs using **well-defined control structures** such as sequence, selection (decision-making), and iteration (loops). It makes programs easier to understand, debug, and maintain.

Algorithm

An algorithm is a step-by-step finite sequence of instructions to solve a problem or perform a task.

Features of a Good Algorithm:

1. **Finiteness** – Must end after a finite number of steps.
2. **Definiteness** – Each step should be clear and unambiguous.
3. **Input** – Zero or more inputs must be provided.
4. **Output** – At least one output should be produced.
5. **Effectiveness** – Steps must be simple, basic, and feasible.
6. **Generality** – Should solve a set of similar problems.

Need for algorithms

1. Algorithms are necessary for solving complex problems efficiently and effectively.

2. They help to automate processes and make them more reliable, faster, and easier to perform.
3. Algorithms also enable computers to perform tasks that would be difficult or impossible for humans to do manually.
4. They are used in various fields such as mathematics, computer science, engineering, finance, and many others to optimize processes, analyze data, make predictions, and provide solutions to problems.

Use of the Algorithms:

Algorithms play a crucial role in various fields and have many applications. Some of the key areas where algorithms are used include:

1. **Computer Science:** Algorithms form the basis of computer programming and are used to solve problems ranging from simple sorting and searching to complex tasks such as artificial intelligence and machine learning.
2. **Mathematics:** Algorithms are used to solve mathematical problems, such as finding the optimal solution to a system of linear equations or finding the shortest path in a graph.
3. **Operations Research:** Algorithms are used to optimize and make decisions in fields such as transportation, logistics, and resource allocation.
4. **Artificial Intelligence:** Algorithms are the foundation of artificial intelligence and machine learning, and are used to develop intelligent systems that can perform tasks such as image recognition, natural language processing, and decision-making.
5. **Data Science:** Algorithms are used to analyze, process, and extract insights from large amounts of data in fields such as marketing, finance, and healthcare.

Benefits of Algorithm:

- Easy to understand.
- Provides a clear solution method.
- Helps in program design.
- Easy to debug and maintain.
- **Accuracy and Precision:**
Algorithms provide precise, consistent, and reliable results because they are based on logical, step-by-step procedures, not subjective human judgment.
- **Efficiency:**
Algorithms are designed to optimize processes and use resources effectively, allowing for quick execution and processing of large datasets.

- **Automation:**

They automate repetitive and routine tasks, freeing humans to focus on more complex and creative activities, thereby increasing productivity and efficiency.

- **Scalability:**

Algorithms can be scaled to handle larger and more complex problems, processing millions or billions of data points effectively.

- **Reproducibility:**

The structured nature of algorithms makes them reproducible, which is vital for research and development across various fields.

- **Clarity and Understanding:**

Algorithms break down complex problems into simpler, logical steps, making the logic clear and easier to understand for others.

Drawbacks of Algorithm:

- Writing complex algorithms is time-consuming.
- Not suitable for very large or complicated problems.
- Requires translation into code before execution.
- **Time-Consuming Development:**

Developing algorithms can be time-consuming, especially for large or complex problems.

- **Complexity in Design:**

Representing complex branching and looping logic within an algorithm can be difficult to design and implement clearly.

- **Potential for Bias:**

Algorithms can inadvertently amplify existing biases present in the data they are trained on, leading to unfair or skewed outcomes.

- **Dependency and Lack of Oversight:**

Heavy reliance on algorithms can create a dependency where users trust automated processes without sufficient understanding, which can be problematic if the algorithm fails or behaves unexpectedly.

- **Security Vulnerabilities:**

Like any software, algorithms can have vulnerabilities that malicious actors might exploit, leading to data breaches and other security issues.

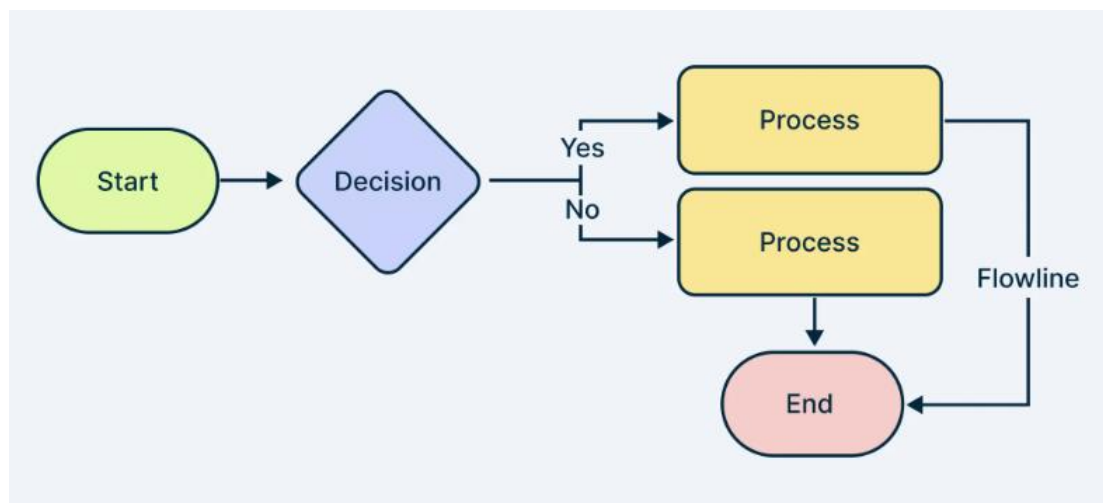
- **Inability to Solve All Problems:**

Algorithms have limitations and cannot effectively address problems that involve subjective judgment or lack clear-cut, deterministic solutions.

Flowcharts:

A flowchart is a diagram that visually represents a step-by-step process, workflow, or system using standardized symbols and arrows to show the sequence of actions and decisions. It provides a shared visual language to document, analyze, and communicate complex information, making it easier to understand, improve, and troubleshoot processes across various fields like business, engineering, and project management.

Example:



Flowcharts are graphical representations of data, algorithms, or processes, providing a visual approach to understanding code.

- Flowcharts illustrate step-by-step solutions to problems, making them useful for beginner programmers.
- Flowcharts help in debugging and troubleshooting issues.
- Flowchart consists of sequentially arranged boxes that depict the process flow.

Since it visually represents an algorithm or workflow, it is easier to interpret and understand. However, to create an effective Flowchart, certain standardized rules must be followed, ensuring clarity and consistency across different professionals worldwide.

Flowchart Symbols

Different types of boxes are used to make Flowchart Symbols. All the different kinds of boxes are connected by arrow lines. Arrow lines are used to display the flow of control. Let's learn about each box in detail.

Symbol	Meaning	Usage Example
Oval (Terminator)	Start / End of program	START, STOP
Rectangle	Process / Task	Calculation: $SUM = A + B$
Diamond	Decision / Condition	IF $A > B$?
Parallelogram	Input / Output	INPUT A, B, PRINT RESULT
Arrow	Flow of Control	Shows direction of steps

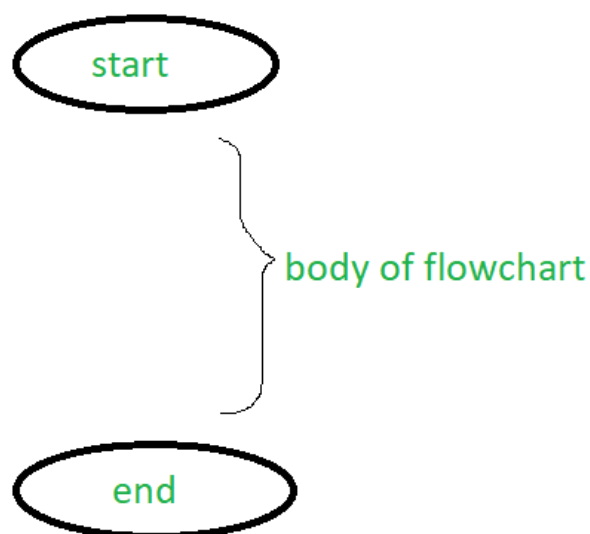
Types of boxes used to make a Flowchart

There are different types of boxes that are used to make Flowcharts. All the different kinds of boxes are connected to one another by arrow lines. Arrow lines are used to display the flow of control. Let's learn about each box in detail.

1. Terminal



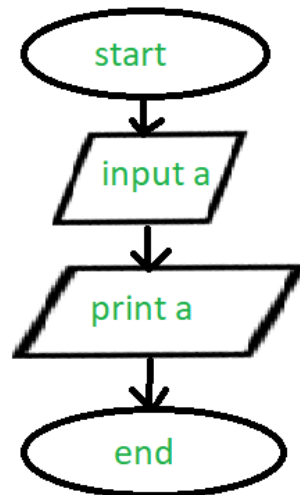
This box is of an oval shape which is used to indicate the start or end of the program. Every Flowchart diagram has an oval shape that depicts the start of an algorithm and another oval shape that depicts the end of an algorithm. For example:



2. Data



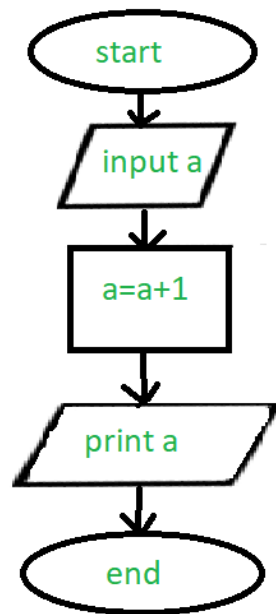
This is a parallelogram-shaped box inside which the inputs or outputs are written. This basically depicts the information that is entering the system or algorithm and the information that is leaving the system or algorithm. For example: if the user wants to input a from the user and display it, the Flowchart for this would be:



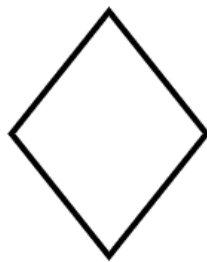
3. Process



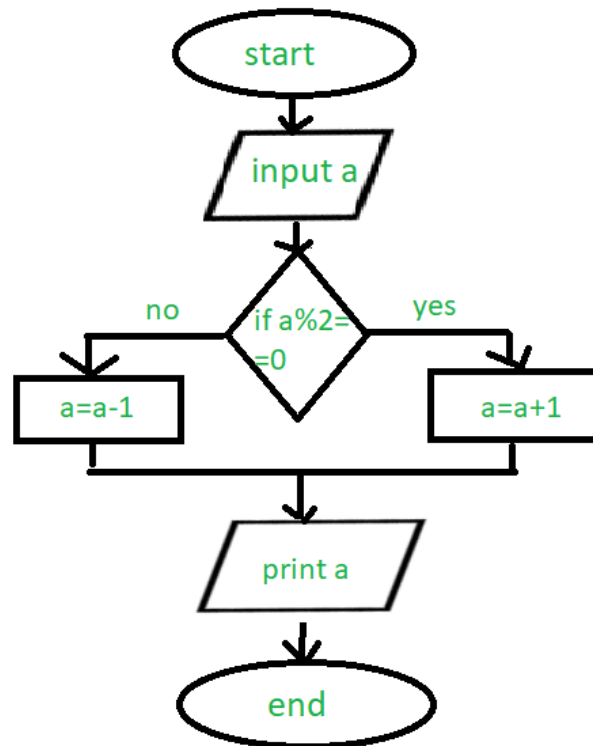
This is a rectangular box inside which a programmer writes the main course of action of the algorithm or the main logic of the program. This is the crux of the Flowchart as the main processing code is written inside this box. For example: if the programmer wants to add 1 to the input given by the user, he/she would make the following Flowchart:



4. Decision



This is a rhombus-shaped box, control statements like if, condition like $a > 0$, etc are written inside this box. There are 2 paths from this one which is "yes" and the other one is "no". Every decision has either yes or no as an option; similarly, this box has these as options. For example: if the user wants to add 1 to an even number and subtract 1 if the number is odd, the Flowchart would be:



5. Flow

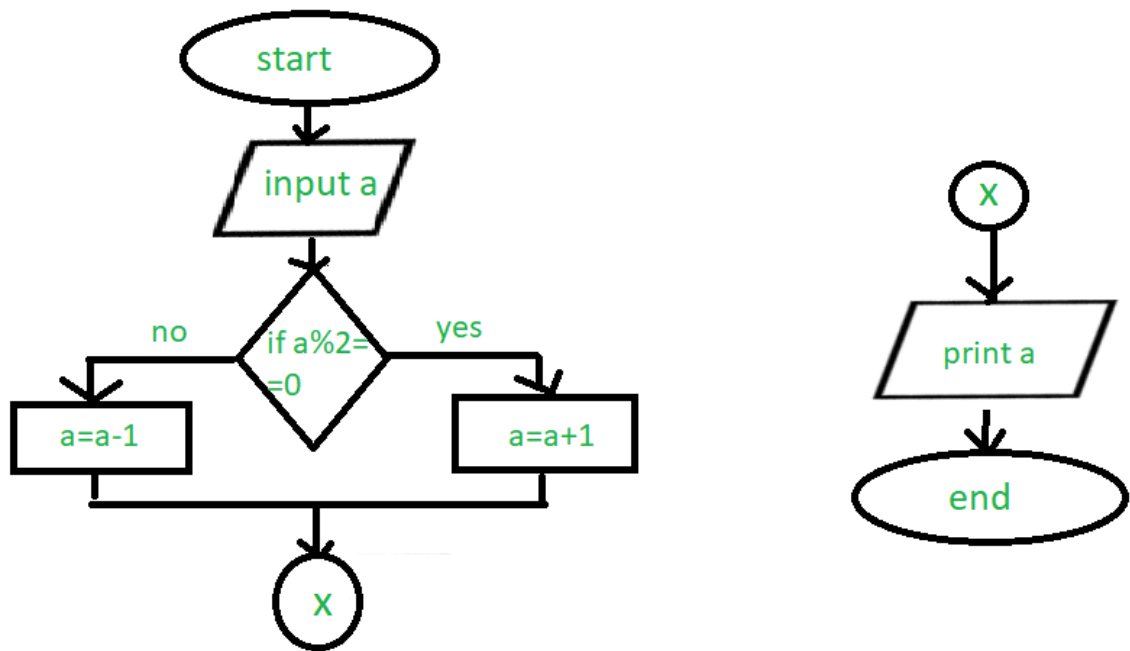


This arrow line represents the flow of the algorithm or process. It represents the direction of the process flow. In all the previous examples, we included arrows in every step to display the flow of the program. Arrows increase the readability of the program.

6. on-Page Reference



This circular figure is used to depict that the Flowchart is in continuation with the further steps. This figure comes into use when the space is less and the Flowchart is long. Any numerical symbol is present inside this circle and that same numerical symbol will be depicted before the continuation to make the user understand the continuation. Below is a simple example depicting the use of On-Page Reference



Uses of Flowcharts in Computer Programming/Algorithms

The following are the uses of a Flowchart:

- It is a pictorial representation of an algorithm that increases the readability of the program.
- Complex programs can be drawn in a simple way using a Flowchart.
- It helps team members get an insight into the process and use this knowledge to collect data, detect problems, develop software, etc.
- A Flowchart is a basic step for designing a new process or adding extra features.
- Communication with other people becomes easy by drawing Flowcharts and sharing them.

When to Use Flowchart?

Flowcharts are mainly used in the below scenarios:

- It is most importantly used when programmers make projects. As a Flowchart is a basic step to make the design of projects pictorially, it is preferred by many.
- When the Flowcharts of a process are drawn, the programmer understands the non-useful parts of the process. So, Flowcharts are used to separate sound logic from the unwanted parts.
- Since the rules and procedures of drawing a Flowchart are universal, a Flowchart serves as a communication channel to the people who are working on the same project for better understanding.

- Optimizing a process becomes easier with Flowcharts. The efficiency of the code is improved with the Flowchart drawing.

Types of Flowcharts

Three types of Flowcharts are listed below:

1. **Process Flowchart:** This type of Flowchart shows all the activities that are involved in making a product. It provides a pathway to analyze the product to be built. It is most commonly used in process engineering to illustrate the relation between the major as well as minor components present in the product. It is used in business product modelling to help understand employees about the project requirements and gain some insight into the project.
2. **Data Flowchart:** It is used to analyze the data; specifically it helps in analyzing the structural details related to the project. Using this Flowchart, one can easily understand the data inflow and outflow from the system. It is most commonly used to manage data or to analyze information to and fro from the system.
3. **Business Process Modeling Diagram:** Using this Flowchart or diagram, one can analytically represent the business process and help simplify the concepts needed to understand business activities and the flow of information. This Flowchart illustrates the business process and models graphically which paves the way for process improvement.

Advantages and limitations of flowcharts

Advantages of Flowchart

- It is the most efficient way of communicating the logic of the system.
- It acts as a guide for a blueprint during the program design.
- It also helps in the debugging process.
- Using Flowcharts we can easily analyze the programs.
- Flowcharts are good for documentation.

Disadvantages of Flowchart

- Flowcharts are challenging to draw for large and complex programs.
- It does not contain the proper amount of details.
- Flowcharts are very difficult to reproduce.
- Flowcharts are very difficult to modify.

Pseudocode:

A **Pseudocode** is defined as a step-by-step description of an algorithm. Pseudocode does not use any programming language in its representation instead it uses the simple English language text as it is intended for human understanding rather than machine reading. Pseudocode is the **intermediate state between an idea and its implementation (code)** in a high-level language.

need for Pseudocode

Pseudocode is an important part of designing an algorithm, it helps the programmer in planning the solution to the problem as well as the reader in understanding the approach to the problem. Pseudocode is an intermediate state between algorithm and program that plays supports the transition of the algorithm into the program.



Pseudocode is an intermediate state between algorithm and program

Writing a pseudocode

How to write Pseudocode?

Before writing the pseudocode of any algorithm the following points must be kept in mind.

- Organize the sequence of tasks and write the pseudocode accordingly.
- At first, establishes the main goal or the aim.
- **Example:**

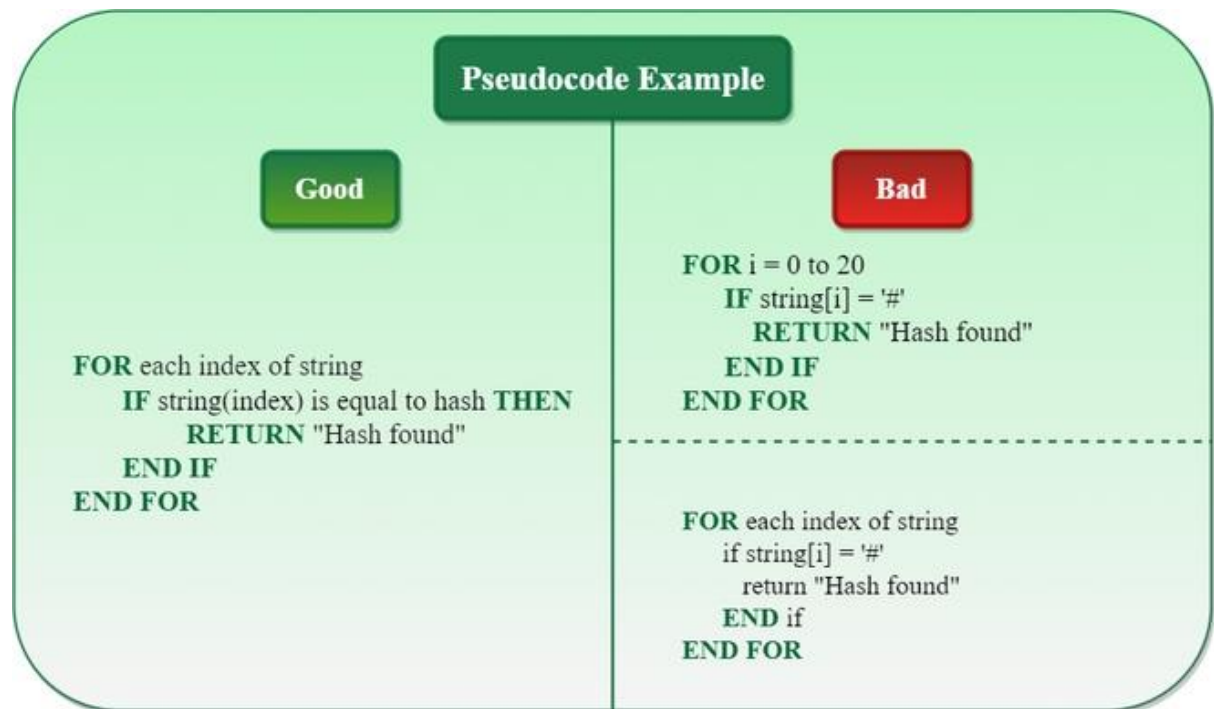
This program will print first N numbers of Fibonacci series.

- Use standard programming structures such as **if-else**, **for**, **while**, and **cases** the way we use them in programming. Indent the statements if-else, for, while loops as they are indented in a program, it helps to comprehend the decision control and execution mechanism. It also improves readability to a great extent.

Example:

```
IF "1"  
    print response  
    "I AM CASE 1"  
IF "2"  
    print response  
    "I AM CASE 2"
```

- Use appropriate naming conventions. The human tendency follows the approach of following what we see. If a programmer goes through a pseudo code, his approach will be the same as per that, so the naming must be simple and distinct.
- Reserved commands or keywords must be represented in **capital letters**. **Example:** if you are writing IF...ELSE statements then make sure IF and ELSE be in capital letters.
- Check whether all the sections of a pseudo code are complete, finite, and clear to understand and comprehend. Also, explain everything that is going to happen in the actual code.
- Don't write the pseudocode in a programming language. It is necessary that the pseudocode is simple and easy to understand even for a layman or client, minimizing the use of technical terms.



Pseudo code Examples:

1. Binary search Pseudo code:

Binary search is a searching algorithm that works only for sorted search space. It repeatedly divides the search space into **half** by using the fact that the search space is sorted and checking if the desired search result will be found in the left or right half.

Example: Given a sorted array **Arr[]** and a value **X**, The task is to find the index at which **X** is present in **Arr[]**.

Below is the pseudocode for Binary search.

BinarySearch(ARR, X, LOW, HIGH)

repeat till LOW = HIGH

MID = (LOW + HIGH)/2

if (X == ARR[mid])

return MID

else if (x > ARR[MID])

LOW = MID + 1

else

HIGH = MID - 1

Difference between Algorithm and Pseudocode

Algorithm	Pseudocode
An Algorithm is used to provide a solution to a particular problem in form of a well-defined step-based form.	A Pseudocode is a step-by-step description of an algorithm in code-like structure using plain English text.
An algorithm only uses simple English words	Pseudocode also uses reserved keywords like if-else, for, while, etc.
These are a sequence of steps of a solution to a problem	These are fake codes as the word pseudo means fake, using code like structure and plain English text
There are no rules to writing algorithms	There are certain rules for writing pseudocode
Algorithms can be considered pseudocode	Pseudocode cannot be considered an algorithm
It is difficult to understand and interpret	It is easy to understand and interpret

A **Flowchart** is pictorial representation of flow of an algorithm.

A Pseudocode is a step-by-step description of an algorithm in code like structure using plain English text.

A Flowchart uses standard symbols for input, output decisions and start stop statements. Only uses different shapes like box, circle and arrow.

Pseudocode uses reserved keywords like if-else, for, while, etc.

This is a way of visually representing data, these are nothing but the graphical representation of the algorithm for a better understanding of the code	These are fake codes as the word pseudo means fake, using code like structure but plain English text instead of programming language
Flowcharts are good for documentation	Pseudocode is better suited for the purpose of understanding

Coding a Program

Coding is the process of **translating an algorithm or pseudocode into a specific programming language** (like C, Python, or Java). It converts logical steps into executable computer instructions.

Steps in Coding:

1. Choose the programming language.
2. Convert the algorithm or pseudocode into program statements.
3. Follow syntax and grammar rules of the language.
4. Test the program for correctness.

Example:

Algorithm: Find the sum of two numbers.

```
START
  INPUT A, B
  SUM = A + B
  PRINT SUM
STOP
```

C Code:

```
#include <stdio.h>
int main() {
    int a, b, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d", sum);
    return 0;
}
```

Documenting a Program

Documentation is the process of **writing explanations and comments** in a program to make it easier to understand, debug, and maintain.

Purpose of Documentation:

- Helps programmers understand code logic.
- Useful for future maintenance and debugging.
- Provides clarity for others who read or modify the program.

Comment Lines:

Used to add explanations inside code without affecting execution.

- **Single-line comment (C, C++, Java):**

```
// This line calculates the sum
```

- **Multi-line comment:**

```
/* This program reads
two numbers and finds their sum */
```

Testing a Program

Testing is the process of **executing a program with the intent to find and fix errors**. It ensures the program works as expected.

Types of Testing:

- **Unit Testing:** Testing individual modules.
- **Integration Testing:** Testing how modules work together.
- **System Testing:** Testing the complete program.

Command lines and Types of Errors

1. Syntax Errors:

- Caused by incorrect use of language rules (missing semicolon, wrong keyword).
- Detected by compiler.
- Example:

```
printf("Hello")    // Missing semicolon
```

2. Logical Errors:

- Occur when logic is wrong; program runs but gives incorrect output.
- Example: Using * instead of +.

3. Runtime Errors:

- Occur while the program is running.
- Example: Division by zero, accessing invalid memory.

Program Design: Modular Programming

Modular programming is a **software design technique** that divides a large program into smaller, independent **modules**, each performing a specific function.

Advantages:

- Easier to debug and test.
- Promotes code reuse.
- Improves readability and organization.
- Allows multiple programmers to work in parallel.

Example: *Library Management System*

Modules can include:

- AddBook () – Adds new books.
- SearchBook () – Searches for books.
- IssueBook () – Issues books to users.
- ReturnBook () – Handles book returns.

```
void AddBook() { /* Code to add a book */ }  
void SearchBook() { /* Code to search a book */ }  
void IssueBook() { /* Code to issue a book */ }  
void ReturnBook() { /* Code to return a book */ }
```

QUESTIONS

2-Mark Questions

1. What is data?
 2. Define algorithm.
 3. List two types of errors.
 4. What is a pseudocode?
 5. Define modular programming.
-

5-Mark Questions

1. Explain the phases of the Program Development Cycle.
 2. Write short notes on the advantages and limitations of flowcharts.
 3. Explain the features of a good algorithm.
-

10-Mark Questions

1. Explain the different phases in the Program Development Cycle.
2. Describe algorithm, flowchart, and pseudocode with examples.
3. Discuss coding, documenting, testing, and modular programming in detail.

UNIT III

Selection Structures:

- A **selection structure** allows a program to make **decisions** and **execute different statements** based on certain **conditions**.
- It is also called a **decision-making structure** or **conditional statement**.
- Selection structures use **relational** and **logical operators** to compare data and determine which path the program should take.

Relational and Logical Operators

Relational operators are the symbols that are used for comparison between two values to understand the type of relationship a pair of numbers shares. The result that we get after the relational operation is a boolean value, that tells whether the comparison is true or false. Relational operators are mainly used in conditional statements and loops to check the conditions in C programming.

Relational Operators in C

Operator Symbol	Operator Name
==	Equal To
!=	Not Equal To
<	Less Than
>	Greater Than
<=	Less Than Equal To
>=	Greater Than Equal To

Types of C Relational Operators

There are a total of 6 relational operators in C language. There are:

1. Equal to operator (==)

The C **equal to** operator (==) is a relational operator that is used to check whether the two given operands are equal or not.

- Equal to operator is a binary operator hence it requires two operands to perform the comparison.

- If the two values are equal, it returns true. Otherwise, it returns false.
- It does not work for strings or arrays.

Syntax

```
operand1 == operand2
```

For example, `5==5` will return true.

2. Not equal to operator (!=)

The C not equal (`!=`) to operator is another relational operator used for checking whether the two given operands are equal or not.

- It is also a binary operator, requiring two operands to perform the comparison.
- It is the exact boolean complement of the `'=='` operator which returns true if the two values are not equal, false otherwise.

Syntax

```
operand1 != operand2
```

For example, `5!=5` will return false.

3. Greater than operator (>)

The greater than operator is a relational operator in C that checks whether the first operand is greater than the second operand or not.

- It is a binary operator.
- If the operand first is greater than the operand2, it returns true. Otherwise, it returns false.
- This operator is used to make decisions or create conditions based on the relative magnitude of two values.

Syntax

```
operand1 > operand2
```

For example, `6>5` will return true.

4. Less than operator (<)

The less than operator is a relational operator in C that checks whether the first operand is lesser than the second operand.

- It is a binary operator.
- If the operand first is less than the operand2, it returns true. Otherwise, it returns false.
- This operator is also used to make decisions or create conditions based on the relative magnitude of two values.

Syntax

```
operand1 < operand2
```

For example, **6<5** will return false.

Note: *The greater than and less than operator are not equal to the complement of each other.*

5. Greater than or equal to operator (>=)

The greater than or equal to the operator is a relational operator in C that checks whether the first operand is greater than or equal to the second operand.

- It is a binary operator.
- If the first operand is greater than or equal to the second operand, it returns true.

Otherwise, it returns false.

Syntax

```
operand1 >= operand2
```

For example, **5>=5** will return true.

6. Less than or equal to the the operator (<=)

The less than or equal to the operator is a relational operator in C that checks whether the first operand is less than or equal to the second operand.

- It is a binary operator.
- If the first operand is less than or equal to the second operand, it returns true.

Otherwise, it returns false.

Syntax

```
operand1 <= operand2
```

For example, **5<=5** will also return true.

Example of Relational Operator in C

```
#include <stdio.h>
```

```
int main()
{
    int a = 10, b = 4;

    // greater than example
    if (a > b)
        printf("a is greater than b\n");
    else
        printf("a is less than or equal to b\n");

    // greater than equal to
    if (a >= b)
        printf("a is greater than or equal to b\n");
    else
        printf("a is lesser than b\n");
}
```

```

// less than example
if (a < b)
    printf("a is less than b\n");
else
    printf("a is greater than or equal to b\n");

// lesser than equal to
if (a <= b)
    printf("a is lesser than or equal to b\n");
else
    printf("a is greater than b\n");

// equal to
if (a == b)
    printf("a is equal to b\n");
else
    printf("a and b are not equal\n");

// not equal to
if (a != b)
    printf("a is not equal to b\n");
else
    printf("a is equal b\n");

return 0;
}

```

Logical Operators

Logical operators are used to **combine two or more conditions**.

Operator	Meaning	Example	Result
and	True if both conditions are true	(x > 0 and x < 10)	True if x is between 1 and 9
or	True if any one condition is true	(x < 0 or x > 100)	True if x outside 0–100
not	Reverses the result	not(x > 5)	True if x ≤ 5

Selecting from Several Alternatives

1. Simple if Statement

Used when there is **only one condition**.

```

if marks >= 50:
    print("Pass")

```

2. if-else Statement

Used when there are **two alternatives**.

```
if marks >= 50:
    print("Pass")
else:
    print("Fail")
```

3. Nested if Statement

When one **if statement is inside another**, it is called a nested if.

```
if marks >= 50:
    if marks >= 75:
        print("Distinction")
    else:
        print("Pass")
else:
    print("Fail")
```

4. if-elif-else Statement

Used when there are **several conditions**.

```
if grade == "A":
    print("Excellent")
elif grade == "B":
    print("Good")
elif grade == "C":
    print("Average")
else:
    print("Fail")
```

Applications of Selection Structures.

Selection structures are used in programming to make decisions based on certain conditions. They allow a program to choose different actions depending on whether a condition is true or false. These structures are essential for controlling the flow of execution in a program.

Applications:

1. **Decision Making:** Used to perform different actions based on specific conditions, such as checking user input or validating data.
2. **Grading System:** Used to assign grades to students based on their marks or performance levels.
3. **Login Verification:** Used to check whether the entered username and password are correct.

4. **Billing System:** Used to calculate the total bill amount based on the quantity or usage, such as electricity or water bills.
5. **Traffic Control System:** Used to control the traffic lights depending on traffic density or timing.
6. **Temperature Control:** Used in automatic systems to turn devices ON or OFF based on temperature levels.
7. **Banking Applications:** Used to check conditions like sufficient balance before withdrawal.

In summary, **selection structures help in controlling the logical flow of programs**, enabling them to make intelligent decisions and respond dynamically to various situations.

Repetition Structures:

- A **Repetition Structure** (also called a **Loop**) allows a set of instructions to be executed repeatedly until a certain condition is met.
- Loops are one of the most important control structures in programming because they make the program efficient by avoiding the need to write the same code multiple times.

Types of Repetition Structures:

1. Counter-Controlled Loop

- A loop that repeats a specific number of times.
- Uses a **counter variable** that increases or decreases with each iteration.
- Example: `for` loop

Syntax (Python Example):

```
for i in range(1, 6):  
    print("Iteration number:", i)
```

Output:

```
Iteration number: 1  
Iteration number: 2  
Iteration number: 3  
Iteration number: 4  
Iteration number: 5
```

Explanation:

- The loop starts from 1 and runs up to 5.
- `i` acts as the counter variable.

2. Condition-Controlled Loop

- The loop continues execution as long as a **condition** is true.
- When the condition becomes false, the loop stops.
- Example: `while` loop

Syntax (Python Example):

```
count = 1
while count <= 5:
    print("Count:", count)
    count = count + 1
```

Output:

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

Explanation:

- The loop continues as long as `count <= 5`.
- The variable `count` is incremented in each iteration.

3. Nested Loop

- A loop inside another loop.
- The **inner loop** executes completely every time the **outer loop** runs once.

Syntax (Python Example):

```
for i in range(1, 4):
    for j in range(1, 4):
        print("i =", i, "j =", j)
```

Output:

```
i = 1 j = 1
i = 1 j = 2
i = 1 j = 3
i = 2 j = 1
i = 2 j = 2
i = 2 j = 3
```

```
i = 3 j = 1
i = 3 j = 2
i = 3 j = 3
```

Explanation:

- The inner loop executes 3 times for every single iteration of the outer loop.

Applications of Repetition Structures:

- Printing series of numbers.
- Calculating the sum or average of numbers.
- Displaying multiplication tables.
- Searching through lists or arrays.
- Performing repetitive calculations.
- Menu-driven programs.

Example Program 1: Display Multiplication Table

```
num = int(input("Enter a number: "))
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Output (for num = 5):

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Repetition structures make programs efficient, concise, and easy to modify. They save time and reduce human errors by automating repeated operations.

Counter Controlled Loops

- A **Counter-Controlled Loop** is a type of repetition structure in which the loop runs a specific number of times.

It uses a **counter variable** that is initialized, tested, and updated each time the loop executes.

- This kind of loop is very common when the **number of iterations is known** in advance — for example, printing numbers from 1 to 10, displaying the first 20 even numbers, etc.

The loop generally follows three steps:

1. **Initialization** – Set the counter to a starting value.
2. **Condition Checking** – Check whether the counter satisfies the loop condition.
3. **Updation** – Increase or decrease the counter each time the loop runs.

Example : Using **while** Loop

```
# Program to print first 5 even numbers
```

```
count = 1          # Counter initialization
number = 2
```

```
while count <= 5:    # Condition checking
    print(number)
    number += 2       # Increment by 2
    count += 1        # Counter update
```

Explanation:

- The counter variable `count` controls how many times the loop runs.
- The loop prints the first 5 even numbers.
- After each iteration, both `count` and `number` are updated.

Advantages:

- The number of iterations is known before execution.
- Easy to use for repetitive tasks like counting or summation.
- Code is clean and simple.

Disadvantages:

- Not flexible for unknown number of repetitions.
- Manual errors can occur if the counter is not properly initialized or updated.

Nested Loops

A nested loop means a loop statement inside another loop statement.

- For a nested loop, the inner loop performs all of its iterations for each iteration of the outer loop.
- If the outer loop is running from $i = 1$ to 5 and the inner loop is running from $j = 0$ to 3. Then, for each value of the outer loop variable (i), the inner loop will run from $j = 0$ to 3.

1. Nested for Loops

Nested for loop refers to any type of loop that is defined inside a 'for' loop.

```
#include <stdio.h>
```

```
int main() {
    int m = 4; // limit for i
    int n = 5; // limit for j

    // Outer loop
    for (int i = 0; i < m; i++) {
        printf("i = %d: ", i);

        // InnerLoop
        for (int j = 0; j < n; j++) {
            printf("%d ", i * n + j);
        }
        printf("\n");
    }

    return 0;
}
```

Output

```
i = 0: 0 1 2 3 4
i = 1: 5 6 7 8 9
i = 2: 10 11 12 13 14
i = 3: 15 16 17 18 19
```

Uses of Nested Loops

- **Printing Patterns:** Nested loops are often used to print complex patterns such as printing shapes, grids, or tables.
- **Searching and Sorting:** Nested loops are used in algorithms that involve searching for or sorting elements like bubble sort, insertion sort, matrix searching etc.
- **Multi-Dimensional Data:** Nested loops are useful when dealing with multidimensional data structures like 2D or 3D arrays, matrices, list of lists.
- **Dynamic Programming:** Nested loops are commonly used in dynamic Programming for solving problems like knapsack problem or longest common subsequence.

Applications of Repetition Structures.

Repetition structures, also known as **loops**, are used in programming to perform a set of instructions repeatedly until a specific condition is met. They help reduce code duplication and make programs more efficient. Repetition structures are widely used in various applications where tasks need to be executed multiple times.

1. Calculating Sums and Averages

Repetition structures are used to calculate the sum or average of a group of numbers. The loop repeatedly adds numbers until all values are processed.

2. Displaying Sequences and Tables

Loops are used to generate and display number sequences, multiplication tables, or repetitive patterns in output.

3. Accepting Repeated Inputs

Repetition structures are helpful when a program needs to accept input from the user multiple times, for example, entering data until a specific value (like 0) is given.

4. Searching and Processing Data

Loops are used to process elements in a list or array, such as searching for an item, finding the largest or smallest value, or counting occurrences.

5. Generating Patterns

Repetition structures are used in programs to print various text, number, or symbol-based patterns like triangles, squares, or pyramids.

6. Performing Repeated Calculations

In scientific and financial applications, loops are used for repeated calculations such as interest computation, simulations, and mathematical operations.

Conclusion:

Repetition structures are essential in programming as they allow tasks to be repeated efficiently, reduce manual effort, and make programs faster, shorter, and easier to understand.

UNIT III – Question Bank

2-Mark Questions

1. What are relational operators? Give two examples.
 2. What are logical operators?
 3. Define selection structure.
 4. What is a loop in programming?
 5. Define nested loop.
-

5-Mark Questions

1. Explain relational and logical operators with examples.
 2. Write short notes on counter-controlled loops.
 3. Differentiate between selection and repetition structures with examples.
-

10-Mark Questions

1. Explain selection structures in detail and discuss how to select from several alternatives with examples.
2. Describe repetition structures with examples of counter-controlled and nested loops.
3. Write a program using selection and repetition structures to find the sum of even numbers between 1 and 50.

UNIT IV

Data:

Data is a collection of raw facts and figures that have no meaning until they are processed to get useful information.

Numeric Data and Character Based

Numeric data represents **numbers** used for arithmetic operations like addition, subtraction, multiplication, and division.

Types of Numeric Data:

1. Integer Data (**int**)

- Used to store whole numbers (without decimal points).
- Example:
 - `int age = 25;`
 - `int year = 2025;`

2. Floating-Point Data (**float / double**)

- Used to store numbers with decimals.
- `float` – single precision
- `double` – double precision
- Example:
 - `float marks = 85.5;`
 - `double pi = 3.14159;`

Uses of Numeric Data:

- For all mathematical calculations.
- Used in counters, loops, statistics, and scientific computing.

Character-Based Data

- Character-based data represents **letters, digits, or symbols** and is stored using the `char` data type.
- Each character is internally stored as an **ASCII value** (American Standard Code for Information Interchange).

Example:

```
char grade = 'A';  
char symbol = '#';
```

Explanation:

- 'A' is stored as ASCII value **65**.
- Characters always use **single quotes** (' ').

Arrays:

- An **array** is a collection of **similar type of data items** stored in **consecutive memory locations**.
- Each element in an array can be accessed by using an **index number**.

Syntax:

```
data_type array_name[size];
```

Example:

```
int marks[5];
```

Here, marks is an array of 5 integers.

One Dimensional Array

A 1D array is a list of elements of the same type arranged in a **single row** or **linear form**.

Declaration:

```
int num[5];
```

Initialization:

```
int num[5] = {10, 20, 30, 40, 50};
```

Accessing Elements:

Each element is accessed using its index starting from 0.

```
printf("%d", num[2]);    // Output: 30
```

Example Program:

```
#include <stdio.h>
int main() {
    int i;
    int marks[5] = {80, 75, 90, 85, 70};
    for(i=0; i<5; i++) {
        printf("Mark %d = %d\n", i+1, marks[i]);
    }
    return 0;
}
```

Memory Representation of 1D Array:

Index	0	1	2	3	4
Value	80	75	90	85	70

Two Dimensional Arrays

A 2D array is an **array of arrays**, used to represent data in **rows and columns**, similar to a matrix.

Syntax:

```
data_type array_name[rows][columns];
```

Example:

```
int marks[2][3] = {  
    {80, 70, 90},  
    {85, 75, 95}  
};
```

Accessing Elements:

```
printf("%d", marks[1][2]); // Output: 95
```

Example Program:

```
#include <stdio.h>  
int main() {  
    int i, j;  
    int marks[2][3] = {{80, 70, 90}, {85, 75, 95}};  
    for(i=0; i<2; i++) {  
        for(j=0; j<3; j++) {  
            printf("%d ", marks[i][j]);  
        }  
        printf("\n");  
    }  
    return 0;  
}
```

Memory Representation of 2D Array:

	Col 0	Col 1	Col 2
Row 0	80	70	90
Row 1	85	75	95

Strings as Arrays of Characters

A **string** is a **sequence of characters** stored in an array and terminated by a **null character** (`'\0'`) to mark the end of the string.

Declaration:

```
char name[10];
```

Initialization:

```
char name[5] = {'J', 'O', 'H', 'N', '\0'};
```

or simply:

```
char name[] = "JOHN";
```

Accessing Characters:

```
printf("%c", name[1]); // Output: O
```

String Example Program:

```
#include <stdio.h>
#include <string.h>
int main() {
    char name[20];
    printf("Enter your name: ");
    gets(name);
    printf("Welcome %s", name);
    return 0;
}
```

Common String Functions (in <string.h>)

Function	Description	Example
strlen(str)	Returns length of string	strlen("HELLO") → 5
strcpy(a,b)	Copies string b into a	strcpy(name, "RAM");
strcat(a,b)	Concatenates (joins) b to a	strcat(s1, s2);
strcmp(a,b)	Compares two strings	Returns 0 if equal

Advantages of Using Arrays and Strings

- Simplifies handling of large amounts of similar data.
- Allows faster data access using index numbers.
- Supports loops for reading and displaying data easily.
- Strings simplify handling of textual information.

In C programming, **numeric and character data** form the foundation for processing information. **Arrays** allow storing multiple values under one name, and **strings** extend this concept to handle textual data efficiently. Understanding these concepts helps in designing structured, efficient, and organized programs.

Questions

2-Mark Questions

1. Define an array.
 2. What is a string in C?
 3. List any two string functions.
 4. What is the difference between numeric and character data?
 5. Define one-dimensional array with an example.
-

5-Mark Questions

1. Explain numeric and character-based data with examples.
 2. Explain the concept of one-dimensional array with syntax and example.
 3. What are strings? Explain with suitable examples.
-

10-Mark Questions

1. Explain in detail about **numeric and character data** and **different types of arrays** with examples.
2. Describe **one-dimensional and two-dimensional arrays**. How are strings represented as arrays of characters? Give examples.

UNIT V

Data Flow Diagrams:

Definition

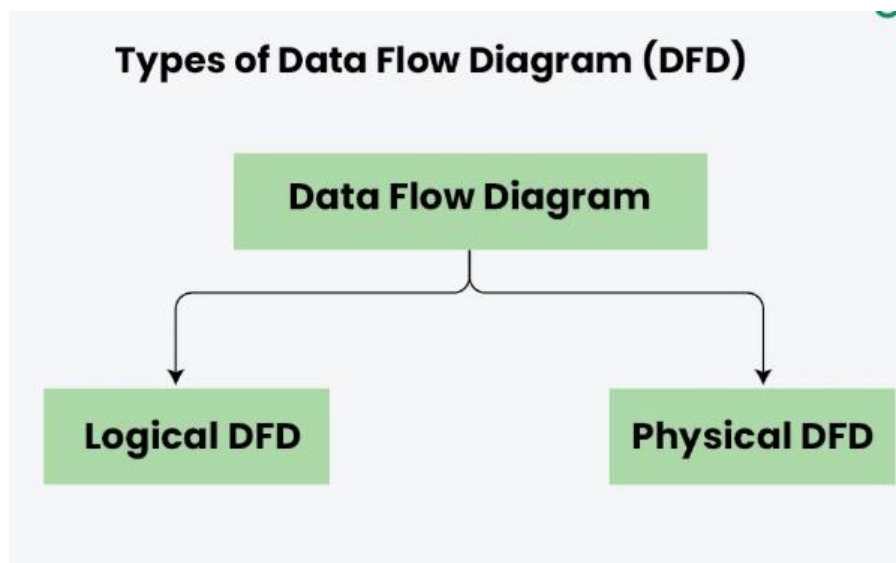
Data Flow Diagram is a **visual representation** of the flow of data within the system. It helps to understand the flow of data throughout the system, from input to output, and how it gets transformed along the way. The models enable **software engineers**, customers, and users to work together effectively during the analysis and **specification of requirements**.

DFD symbols and types of DFDs

Symbol	Meaning	Example
 Rectangle	External Entity – source/destination of data	Customer, Bank
 Circle / Rounded Rectangle	Process – task or activity	Calculate Bill
 Open Rectangle	Data Store – where data is stored	Database
 Arrow	Data Flow – shows movement of data	Order → Payment

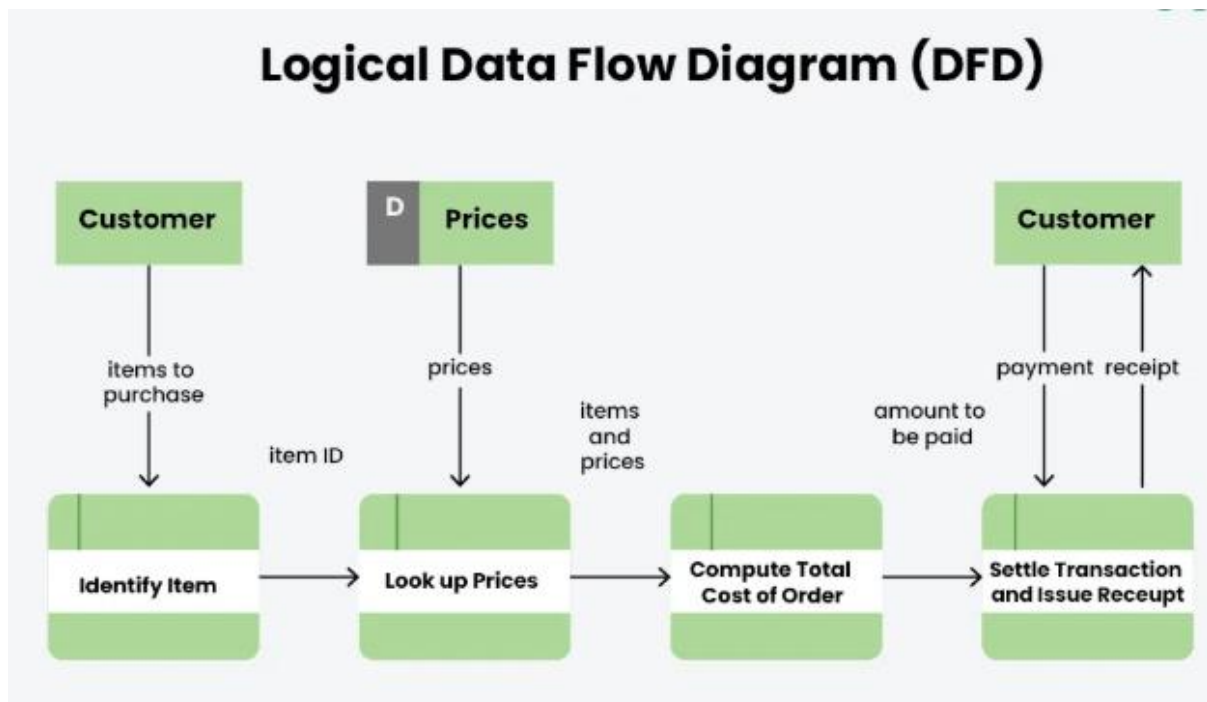
Types of Data Flow Diagram (DFD)

DFDs can be classified into two main types, each focusing on a different perspective of system design:



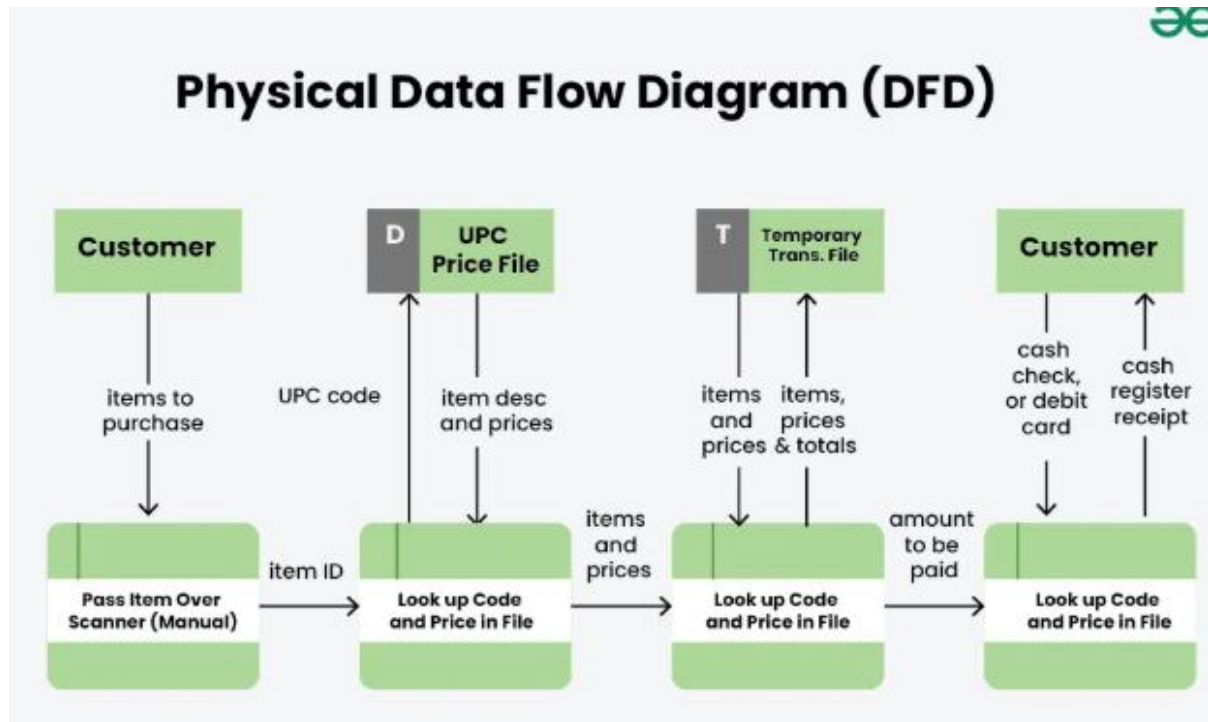
1. Logical Data Flow Diagram (DFD)

- The **Logical Data Flow Diagram** mainly focuses on the **system process**. It illustrates how data flows in the system. Logical Data Flow Diagram (DFD) mainly focuses on high level processes and data flow without diving deep into technical implementation details.
- **Logical DFDs** is used in various organizations for the smooth running of system. Like in a Banking software system, it is used to describe how data is moved from one entity to another.



2. Physical Data Flow Diagram

- Physical data flow diagram shows **how the data flow is actually implemented** in the system. In the Physical Data Flow Diagram (DFD), we include additional details such as data storage, data transmission, and specific technology or system components.
- **Physical DFDs** are more detailed and provide a closer look at the **actual implementation** of the system, including the hardware, software, and physical aspects of data processing.



Program Modules:

A program module is a separate and independent part of a program that performs a specific function.

Modules make programs easier to develop, debug, and maintain.

Subprograms

A subprogram is a smaller program that performs a specific task.

There are two main types:

1. **Procedures** – Do not return a value.
2. **Functions** – Return a value.

Value and Reference parameters

Value parameters

- The value parameters copy the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.
- This is the default mechanism for passing parameters to a method. In this mechanism, when a method is called, a new storage location is created for each value parameter.

- The values of the actual parameters are copied into them. Hence, the changes made to the parameter inside the method have no effect on the argument.

Reference Parameters

- A reference parameter is a reference to a memory location of a variable. When you pass parameters by reference, unlike value parameters, a new storage location is not created for these parameters. The reference parameters represent the same memory location as the actual parameters that are supplied to the method.
- You can declare the reference parameters using the ref keyword.

Example in Python:

```
# Value Parameter Example
def add(a, b):
    return a + b

result = add(5, 3)
print("Sum =", result)

# Reference Parameter Example
def modify_list(items):
    items.append(100)

numbers = [10, 20, 30]
modify_list(numbers)
print("Modified List:", numbers)
```

Scope of a variable-Functions

- **Scope of a variable** defines the region of the code where a particular variable can be referenced or used. Scope holds the current state of variables along with their respective values. Scope of a variable is depends on where the value is declared in the program. Once, a variable is declared, we can access and manipulate its data according to our needs.
- Understanding how the variable's value is accessed and used within its scope is essential to find out the program's behavior.
- Once the program exits the scope where a variable is defined, the memory allocated to that variable is usually released, making the variable inaccessible.
- This ensures that resources are efficiently managed and prevents accidental access or modification of the variable's data outside its intended scope.

Scope of a variable in different programming languages

1. Scope of a variable in C

Variables in C can have two types of scopes:

- **Local Scope or Block Scope:** The variables declared within the local scope are called local variables. Local variables are visible in the block they are declared in and other blocks nested inside that block.
- **Global Scope or File Scope:** The variables declared in the global scope are called global variables. Global variables are visible in every part of the program.

Example:

```
x = 10  # Global variable

def show():
    y = 5  # Local variable
    print("Inside function:", x + y)

show()
print("Outside function:", x)
```

Recursion. Files: File Basics

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.

- A recursive algorithm takes one step toward solution and then recursively call itself to further move. The algorithm stops once we reach the solution.
- Since called function may further call itself, this process might continue forever. So it is essential to provide a base case to terminate this recursion process.

Recursion is a technique where a function calls itself until a base condition is met.

Example: Factorial Program

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print("Factorial of 5 =", factorial(5))
```

Output:

Factorial of 5 = 120

A **file** is a collection of data stored permanently on a storage device.

File Operations:

1. **Create / Write a file**
2. **Read a file**
3. **Modify a file**

Creating and reading a sequential file- Modifying Sequential Files.

Creating and Reading a Sequential File

Example:

```
# Create or Write into a File
f = open("student.txt", "w")
f.write("Name: Subraja\n")
f.write("Department: CSE\n")
f.close()

# Read from the File
f = open("student.txt", "r")
print(f.read())
f.close()
```

Output:

```
Name: Subraja
Department: CSE
```

Modifying a Sequential File

Example:

```
# Modify contents of the file
f = open("student.txt", "r")
data = f.read()
f.close()

# Replace text
data = data.replace("CSE", "ECE")

# Write the modified data back
f = open("student.txt", "w")
f.write(data)
f.close()

print("File modified successfully!")
```

Summary Table

Topic	Description	Example
DFD	Shows how data moves in a system	Context & Level 1 diagrams
Program Module	Independent part of a program	Function or Subprogram
Parameters	Value and Reference types	add(a, b), modify_list(lst)
Recursion	Function calls itself	Factorial
File Handling	Create, Read, Modify files	open(), read(), write()

UNIT V – QUESTION BANK

2-Mark Questions

1. What is a Data Flow Diagram (DFD)?
2. List any two symbols used in DFDs.
3. What is a subprogram?
4. Define recursion.
5. What is a sequential file?

5-Mark Questions

1. Explain the different symbols used in Data Flow Diagrams.
2. Differentiate between call by value and call by reference.
3. Write short notes on file operations – creating and reading a sequential file.

10-Mark Questions

1. Explain in detail the types of Data Flow Diagrams with neat examples.
2. Discuss subprograms, parameters, variable scope, and recursion with suitable examples.
3. Describe file handling concepts in detail — creating, reading, and modifying sequential files with examples.